KOLMOGOROV-ARNOLD NETWORKS FOR ENHANCED MOLECULAR EMBEDDINGS - KAN WE DO BETTER?

Noah Sawyer

Supervisor: Doctor Sarat Moka

School of Mathematics and Statistics
UNSW Sydney

August 2026

Submitted in partial fulfilment of the requirements of the degree of
Master of Mathematics

Plagiarism statement

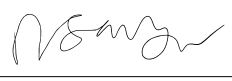
I declare that this thesis is my own work, except where acknowledged, and has not been submitted for academic credit elsewhere.

I acknowledge that the assessor of this thesis may, for the purpose of assessing it:

- Reproduce it and provide a copy to another member of the University; and/or,
- Communicate a copy of it to a plagiarism checking service (which may then retain a copy of it on its database for the purpose of future plagiarism checking).

I certify that I have read and understood the University Rules in respect of Student Academic Misconduct, and am aware of any potential plagiarism penalties which may apply.

By signing this declaration I am agreeing to the statements and conditions above.

Signed:  Date: 07/08/2026

Acknowledgements

I would like to thank my supervisor, Dr Moka, for the refinement and advice provided throughout the writing and design of this thesis. I am especially grateful for the freedom given to me throughout the project, where I have learned invaluable lesson and skills as a result of it.

I thank my family and friends for their support throughout this thesis and my study more broadly. I would never have reached this point without those around me.

Of course, thank you Courtney for putting up with the long nights and weekends, and the constant checking of running code. Your patience has let me create something I am immensely proud of, and I am forever grateful.

Abstract

Graph Neural Networks (GNNs), and Graph Autoencoders (GAEs) in particular, have become a central tool for molecular property prediction, offering a way to compress molecular graphs into low-dimensional latent embeddings that can support many downstream drug discovery tasks with reduced computational burden. Kolmogorov-Arnold Networks (KANs) have recently been proposed as an alternative to conventional Multilayer Perceptrons (MLPs), with claims of improved accuracy and robustness. KAN modules have since been incorporated into GNNs for molecular property prediction, with state-of-the-art results having been reported. However, there does not yet exist an application of KANs to the GAE architecture specifically. The Contrastive Kolmogorov-Arnold Network Graph Autoencoder (CKANGA) addresses this gap. CKANGA is a GAE consisting of Fourier-basis KAN modules in place of MLP layers. CKANGA uses a loss consisting of both graph-level feature and Laplacian-spectrum topology reconstruction terms as well as a batch-wise contrastive loss. Three CKANGA models of subsequently reducing complexity were evaluated against a MLP-based GAE (CMLPGA) on the BACE, BBBP, HIV and Tox21 MoleculeNet benchmarks. The largest CKANGA model, with over two million parameters, consistently underperformed the CMLPGA baseline. Progressively reducing the model complexity resulted in a general improvement in performance, with the smallest model ultimately exceeding the CMLPGA baseline across nearly all datasets and context sizes. This rudimentary variant approached existing state-of-the-art results on BBBP. Speculative explanations of this relationship are examined. These findings suggest that CKANGA may have a niche as compact, parameter-efficient alternatives to MLP-based GNNs.

Contents

Chapter 1

Introduction

The study of molecular interactions underpins much of modern pharmacological and biomedical research, spanning toxicological profiling, the identification of novel drug candidates, and the elucidation of the mechanistic processes that govern biological function [40]. Because a molecule's interaction with a biological target is dictated entirely by the arrangement and properties of its constituent atoms and bonding arrangement, a sufficiently detailed understanding of molecular structure can, in principle, allow such interactions to be predicted. In practice, however, exhaustively modelling these interactions from fundamental physical laws is intractable for all but the simplest molecules and targets, particularly given that only a minority of proteins possess experimentally confirmed structures [62]. Machine learning offers a natural alternative, providing reasonable predictive performance for molecular interaction outcomes, even where underlying structures and mechanisms are not fully understood.

Given the naturally graphical structure of molecules, atoms as nodes and bonds as edges, Graph Neural Networks (GNNs) have become a dominant approach to molecular embedding and property prediction [9]. Among the main GNN architectures, Graph Autoencoders (GAEs) are of particular practical interest. By compressing a molecular graph into an information-preserving latent embedding, a single pretrained GAE can support many lightweight downstream predictive modules, rather than requiring an entirely new GNN to be trained for every property of interest [28]. In the context of large-scale drug screening, where numerous properties must be evaluated for each candidate molecule [30], this substantially reduces the computational burden of the screening pipeline relative to training a dedicated GNN for each property.

Separately, recent years have seen renewed interest in the Kolmogorov-Arnold Representation Theorem as a foundation for neural network design. Formalised into the modern Kolmogorov-Arnold Network (KAN) architecture by Liu et al. in 2024 [42], KANs replace the fixed activation functions and linear weights of the MLP with learnable univariate functions, with the original authors reporting improved accuracy and robustness relative

to equivalently sized Multilayer Perceptrons (MLPs). These claims have since been contested on several grounds, including the architecture's genuine computational cost and the dimension-independence of the error bound [25], as well as reports that KANs underperform MLPs under matched computational budgets [72]. Nonetheless, KAN modules have since been incorporated into Graph Convolutional and Graph Attention Networks for molecular property prediction, with reports of state-of-the-art results across a range of molecular benchmarks [36].

Despite this precedent for performance gains in GCN and GAT architectures, no prior work has applied KAN modules to the GAE architecture. This represents a meaningful gap in the literature. Given the practical value of a GAE's reusable, pretrained latent embedding for downstream molecular property prediction, an enhancement to the quality of that embedding space would carry outsized practical gains relative to equivalent performance enhancements realised in single-task GCN and GAT architectures.

This thesis addresses that gap through the Contrastive Kolmogorov-Arnold Network Graph Autoencoder (CKANGA), a GAE architecture in which the node embedding, message passing, readout and decoder modules are constructed from Fourier-basis KAN layers, trained using a combined reconstruction and contrastive loss. The central aim of this work is to determine whether such a KAN-based GAE can produce an enhanced molecular embedding space, relative to a conventional MLP-based GAE, evidenced through improved downstream predictive performance.

The remainder of this thesis is structured as follows. Chapter 2 provides the theoretical background necessary for this work, covering the Kolmogorov-Arnold Representation Theorem and its development into the KAN architecture [42], alongside an overview of molecular embedding and graph neural networks. Chapter 3 surveys existing applications of KANs to GNNs [36] and motivates the specific focus of this thesis on GAEs. Chapter 4 details the CKANGA architecture, along with the data processing, benchmarking [67] and hyperparameter tuning procedures used to evaluate it. Chapter 5 presents the resulting performance of CKANGA and its variants against a comparably specified MLP-based autoencoder across four MoleculeNet benchmark datasets. Chapter 6 discusses these results, exploring several possible explanations for the relationship observed between model complexity and performance. Finally, Chapter 7 concludes the thesis with a summary of its findings and their implications for future applications of KAN-based architectures in molecular embedding.

Chapter 2

Preliminaries

2.1 Kolmogorov Arnold Representation Theorem

The Kolmogorov-Arnold Representation theorem, originally published by Andrey Kolmogorov [32] and later built on by Vladimir Arnold [6] [7], states that any multivariate continuous function $f : [0,1]^n \to \mathbb{R}$ can be represented by a superposition of single-variable continuous functions. The original theorem is given explicitly in Theorem 2.1.1.

Theorem 2.1.1 (Kolmogorov Arnold Representation Theorem)**.** *For any multivariate continuous function $f : [0,1]^n \to \mathbb{R}$ and for $x \in \mathbb{R}^n$, there exists univariate continuous functions $\phi_{j,i} : [0,1] \to \mathbb{R}$ and $\Phi_j : \mathbb{R} \to \mathbb{R}$ such that:*

$$f(x) = \sum_{j=0}^{2n} \Phi_j \left(\sum_{i=1}^{n} \phi_{j,i}(x_i) \right)$$

The Kolmogorov Arnold Representation Theorem (KART) provides a mechanism for reframing a suitable multivariate continuous function into the composition and addition of finitely many univariate continuous functions. For an n-dimensional function, it can be exactly represented using $(2N+1)(N+1)$ univariate functions, namely $2n+1$ 'outer' functions (Φ_j) and $(2n+1)N$ 'inner' functions $(\phi_{j,i})$.

The conditions required for these univariate functions have developed since Kolmogorov's original paper, which only outlined continuity for both the inner and outer functions.

George Lorentz proved that the outer functions of the representation can be homogenised into a single function [44]. David Sprecher later built upon this, showing that the inner functions may be decoupled from the outer sum, instead moving the 'link' to some multiplicative constant [58]. The combination of these alterations is shown in Theorem 2.1.2.

Theorem 2.1.2 (Lorentz-Sprecher alternate theorem)**.** *For any multivariate continuous function $f : [0,1]^n \to \mathbb{R}$ and for $x \in \mathbb{R}^n$, there exist rationally independent constants λ_j,*

and univariate continuous functions $\phi_i : [0,1] \to \mathbb{R}$ *and* $g : \mathbb{R} \to \mathbb{R}$ *such that:*

$$f(x) = \sum_{j=0}^{2n} g\left(\sum_{i=1}^{n} \lambda_j \phi_i(x_i)\right)$$

Buma Fridman later showed that the inner functions adhere to lipschitz continuity [17], however Anatoli Vitushkin, Gennadi Henkin [66] and Robert Kaufman [29] all showed that these inner functions could not be coerced into continuous differentiability. Vitushkin and Henkin showed that the representation does not generalise to all continuous multivariate functions if smoothness is required in the inner functions[66]. Kaufman demonstrated that not all continuous functions can be represented as the linear superpositions of continuous functions[29], like those of the inner functions in KART.

When originally synthesised by Kolmogorov, KART was little more than a footnote in wider mathematics. George Lorentz, in the same publication building upon Kolmogorov's original work [44], lamented "One wonders whether Kolmogorov's theorem can be used to obtain positive results of greater depth". While it demonstrated that there existed a finite univariate decomposition of every multivariate function, the limited conditions on these univariate functions made any practical applications exceedingly limited. In foundational neural network research, the necessity to represent complex functions through a series of simpler ones briefly drew KART back into the limelight. Originally introduced within the field of neural networks by Robert Hecht-Nielsen in 1987 [23], KART demonstrates, at least theoretically, that any continuous function of multiple variables can be exactly captured in a 2 layer network (relating to the inner and outer functions). In the same year the earliest versions of the Universal Approximation Theorem (UAT) were published, Federico Girosi and Tomaso Poggio published a 1989 paper titled *"Representation Properties of Networks: Kolmogorov's Theorem Is Irrelevant"* [21]. As the name suggests, the authors posit that the poor behaviours of the constituent functions within a Kolmogorov Arnold representation make it useless for network representation. The primary argument of Girosi and Poggio hinges on two main points; the earlier works of Vitushkin and Henkin [66] revealing the lack of differentiability conditions on the inner functions (even if the represented function is differentiable), and the lack of parameters in the theorem. This second point demonstrates that unlike linear weight matrices, for example, whose values can be updated through learning, the inner functions of KART do not consist of finitely many parameter values, instead requiring the functions themselves to be updated. These two caveats were thought by the two authors to render Kolmogorov's theorem entirely irrelevant to the field of network learning.

This view, however, would not be left unchallenged for long. Two years later, in 1991, Vèra Kůrková provided a direct counter to Girosi and Poggio, in her paper titled *"Kol-*

mogorov's Theorem Is Relevant" [33]. Arguing along the lines of practical use cases rather than theoretical implementation, Kůrková showed that both the non-smooth behavior of the inner functions and the lack of updatable parameters could both be overcome if the equality in KART was instead treated as an approximation. Kůrková had prior work focusing on staircase-like functions, which are functions consisting of affine transformations on $\mathbb{R}$ followed by a sigmoidal functions. We know know this to be representation of a multilayer perceptron (MLP), with the sigmoidal function acting as the activation function. Kůrková showed that the constraints on the inner and outer functions determined by Sprecher and Lorentz meant that they could be arbitrarily approximated using these staircase-like functions[34], as outlined in Theorem 2.1.3.

Theorem 2.1.3 (Kůrková's approximation theorem)**.** *For $n \geq 2$, Let $f : [0,1]^n \to \mathbb{R}$ be a continuous function and $\epsilon > 0$. Then, $\exists k \in \mathbb{N}$ and sigmoidal type staircase functions $\psi_i, \phi_{i,j}$ such that for any $\mathbf{x} \in [0,1]^n$,*

$$\left| f(\mathbf{x}) - \sum_{j=1}^{k} \psi_j \left(\sum_{i=1}^{n} \phi_{i,j}(x_i) \right) \right| < \epsilon$$

Kůrková's contribution was one of the first to demonstrate the relevancy of KART to network learning. The staircase-like functions, which are equivalent to our contemporary definitions of a multilayer perceptron, consisted parameter that could be updated through backpropagation. Combined with their now proven ability to approximate the poorly behaved inner and outer functions, Kůrková was able to dispel both of the primary concerns of Girosi and Poggio's earlier paper.

Despite these advances, the applications of KART to neural networks was limited for many decades. Kůrková's work, whilst important in confirming the relevancy of KART to the field, was overshadowed by the foundational papers for the Universal Approximation Theorem, which were published around the same time. In Kenichi Funahashi's *"On the approximate realization of continuous mappings by neural networks"*, the author demonstrates the universal approximation of a function using a neural network with sigmoidal activation of any depth greater than two, specifically highlighting KART as a proof of the case for 2-layer networks [18].

Notably, with increasing computational resources, the nature of network learning shifted focus onto practical MLP architectures. KART is an analytically interesting approach to network representation, yet the nature of the constituent functions does not lend to practical implementation. Instead, similarly to Kůrková's work, most uses of KART from 2000 up until the mid 2020's were analysis regarding MLPs. A number of these papers from the 2000's and 2010's are effectively summarised in Johannes Schmidt-Hieber's *"The*

Kolmogorov-Arnold Representation Theorem Revisited" [53]. One such example is Jurgen Braun and Michael Greibel, who introduced several alterations including removing the restricted domain of the inner functions, and replacing the outer functions with a single continuous function [12], outlined in Theorem 2.1.4.

Theorem 2.1.4 (Braun-Griebel alternate theorem)**.** *For $n \geq 2$, There are real numbers a, b_i, c_j and a continuous, monotone $\psi : \mathbb{R} \to \mathbb{R}$ such that for any multivariate continuous function $f : [0,1]^n \to \mathbb{R}$ and for $x \in \mathbb{R}^n$, there exists a univariate continuous function $g : \mathbb{R} \to \mathbb{R}$ such that:*

$$f(x) = \sum_{j=0}^{2n} g\left(\sum_{i=1}^{n} b_i \psi(x_i + ja) + c_j\right)$$

It was analytical advances such as this that sustained the relevance of KART, however it was not until these equations were translated into practical, experimental models that the theorem would garner significantly more intrigue.

2.2 Kolmogorov Arnold Networks

2.2.1 Network foundations

In 2003, a short and relatively unknown paper was published that would fundamentally shift the applications of KART in network learning. Boris Igelnik and Neel Parikh's *"Kolmogorov's Spline Network"* proposed a novel architecture implementing KART into a neural network. Arguing that, in practice, the target multivariate function for approximation will almost always be continuously differentiable with a bounded gradient, the two constructed an approximation function derived from the Lorentz-Sprecher form in Theorem 2.1.2. The approximation, seen in Theorem 2.2.1, utilises cubic splines for both the inner and outer functions, producing a network that can converge to the true function through increasing the width of the second layer.

Theorem 2.2.1 (Kolmogorov Spline Network)**.** *For any multivariate Lipschitz and continuously differentiable function $f : [0,1]^n \to \mathbb{R}$ and for $x \in \mathbb{R}^n$, there exists, for a given $N \in \mathbb{N}$ some network function f_N with spline functions $s(\cdot, \gamma^n)$ and $s(\cdot, \gamma^{ni})$ with spline parameters γ^n, γ^{ni}, rationally independent constants λ_j, given as:*

$$f_N(x) = \sum_{n=1}^{N} s\left(\left(\sum_{i=1}^{n} \lambda_j s(x_i, \gamma^{ni})\right), \gamma^n\right)$$

such that:

$$\|f - f_N\| = O\left(\frac{1}{N}\right)$$

This paper was the first, to the author's knowledge, to leverage KART in producing a network with neither linear layers nor activations. The cubic splines used could, theoret-

ically, provide approximation to a large range of functions, and had learnable parameters that could be effectively updated using a recursive algorithm. Yet this paper remained a purely theoretical implementation of the architecture; The algorithm for updating the parameters was unwieldy, especially in comparison to the back-propagation algorithms in the MLPs at the time. Moreover, while the convergence of the error term $\|f - f_N\|$ is given, the two were unable to provide an error bound, meaning the network may take an exceptionally large N to produce suitable approximations. The architecture would see a resurgence in the early 2020s, with multiple papers practically implementing more advanced versions of the architecture, utilising B-splines [14] [35] or more exotic functions, such as the 'Floor Exponential Step' [54] or 'Sine/arcsine' [71]. Within these papers, major developments were made. Ming-Jun Lai et.al. and Zuowei Shen both identified their KART-based architectures as ignorant to the Curse of Dimensionality (COD)[35] [54]. COD is the tendency for neural network to perform worse as the input dimension size increases, which is an extremely relevant behaviour, as high-dimensional datasets often seen in practical applications. The ability to overcome this issue was therefore very notable.

The development of KART-based neural networks eventually reached a crescendo in early 2024, when Ziming Liu et.al. published their seminal paper *"KAN: Kolmogorov–Arnold Networks"* [42]. This paper outlined the Kolmogorov Arnold Network (KAN), an architecture using a B-Spline-based model similar to Lai [35], but with significantly more rigour in both theoretical properties and practical implementations. The paper introduces KANs as potential alternatives to MLPs, illustrating the similarities between KART and the UAT as theoretical groundings, whilst highlighting the greater accuracy, robustness and interpretability of KANs. Liu et. al. were certainly not the first to apply KART to neural networks, as the preceding sections have detailed, however the paper has become the most visible in the field, garnering over four thousand citations as of June 2026. One of the key innovations in the paper is the relative movement away from strict theoretical boundaries to take a more pragmatic and optimistic approach. Using MLPs in practice, the UAT creates a basis for 'soundness' in forming a model; It is known that making a single layer model can theoretically create a suitable approximation, however there are exceptionally few cases where a single layer is even attempted, with deep, multilayer networks being chosen, even if the UAT stats that they are not needed. Liu et.al. took a similar stance for KART-based networks, building off the variable outer layer size Igelnik and Parikh's Kolmogorov Spline Network [27] and simply using any size width for the function layers. On top of this, the two layer network representation of KART is also ignored, meaning that networks can have significantly more depth. The architecture is outlined in Definition 2.2.2.

Definition 2.2.2 (Kolmogorov Arnold Network)**.** *Let the size of the l^{th} layer be n_l. For*

each layer l, there are $n_l n_{l+1}$ functions $\phi_{l,j,i}$ mapping all nodes i in l to all nodes j in $l+1$. The value of node j in layer $l+1$ is given as:

$$x_{l+1,j} = \sum_{i=1}^{n_l} \phi_{l,j,i}(x_{l,i})$$

Equivalently, we can write this in matrix notation:

$$\boldsymbol{x_{l+1}} = \begin{pmatrix} \phi_{l,1,1}(\cdot) & \phi_{l,1,2}(\cdot) & \cdots & \phi_{l,1,n_l}(\cdot) \\ \phi_{l,2,1}(\cdot) & \phi_{l,2,2}(\cdot) & \cdots & \phi_{l,2,n_l}(\cdot) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1}(\cdot) & \phi_{l,n_{l+1},2}(\cdot) & \cdots & \phi_{l,n_{l+1},n_l}(\cdot) \end{pmatrix} \boldsymbol{x_l} = \boldsymbol{\Phi_l x_l}$$

The output of a Kolmogorov Arnold Network with L layers can therefore be given as:

$$KAN(\boldsymbol{x}) = (\boldsymbol{\Phi_{L-1}} \circ \boldsymbol{\Phi_{L-2}} \circ \cdots \circ \boldsymbol{\Phi_0}(\boldsymbol{x}))$$

The original KART in Theorem 2.1.1 is a specific instance of this network, consisting of 2 layers with widths n and $2n+1$ respectively.

The 'activation' functions ϕ are set as a weighted combination of a 'residual' $b(x) = \text{silu}(x)$ and a k-degree B-Spline on G intervals $\text{spline}(x) = \sum_{i=1}^{G+k} c_i B_i(x)$ with the Spline polynomials $B_i(x)$, which is given in the paper as:

$$\phi(x) = w_b(x) + w_s \text{spline}(x)$$

These spline functions provide arbitrary approximation of a univariate functions, in turn bounding the approximation error of KANs, seen in Theorem 2.2.3.

Theorem 2.2.3 (KAN error bound)**.** *For some constant W dependant on f, there exists k-degree B-Splines $\phi_{l,i,j}$ such that, for a KAN with gridsize G:*

$$\|f - \text{KAN}_G\|_{C^m} \leq \frac{W}{G^{k+1-m}}$$

Importantly, the accuracy of the KAN can be indefinitely improved upon by extending the gridsize of the splines G, at the cost of computational complexity. The number of parameters required for a KAN with maximum width N and L layers is $O\left(N^2 L(G+k)\right) \approx O\left(N^2 L(G)\right)$. Comparing this to an MLP, with $O\left(N^2 L\right)$, the KAN appear to scale worse, however the authors rebut this, stating *"KANs usually require much smaller N than MLPs"*. Theorem 2.2.3 also highlights the possibility for KANs to overcome the COD, as the error bound has no dependence on the dimension of the input. However, it is important

to note that the constant C is dependent on f, meaning that dimensional influences could still be present, as noted in the paper. The paper also relies on the assumption that f actually has a smooth representation with over a continuous space KAN_∞, which may not be true. These assumptions also lead to the conclusion that KANs have a neural scaling of order $k+1$ (Meaning, for N neurons, the test loss is proportional to $\frac{1}{N^{k+1}}$. In comparison, traditional MLP-based networks require potentially exponential growth in the number of neurons to maintain the same error as input dimension increases [39]. Neural scaling is also typically linear with the number of neurons in an MLP, with test loss proportional to $\frac{1}{N}$ [45]. In practice, the authors set $k = 3$ to utilise cubic splines, giving the reason *"too high k might be too oscillatory, leading to optimization issues."* [42]. Another key claim is the "interpretability" of KANs. In the context of Mathematics and Physics, it is often the goal to determine explicit formulae from data. By first pruning a trained KAN (using a technique dubbed 'sparsification' by the authors), then utilising auto-symbolic regression, a symbolic estimation of its learned splines can be obtained, providing an exact formula estimate. The key is the expressivity of the splines, meaning a single 'edge' of the network can be associated with a similar symbolic formula. ReLU-based MLPs, due to the nature of their activation on linear weights, are significantly harder to conduct auto-symbolic regression on. As most symbolic representations are continuous functions, such as trigonometric or exponential expressions, regression against non-smooth piecewise linear functions of the ReLU MLP exhibits poor extrapolation capability outside of the testing set [31]. Liu et.al. substantiate these results with practical implementations of KANs to examples in both mathematics and physics. In both cases, KANs, with order of magnitude fewer parameters, performed better than MLPs, and symbolic representations close to the underlying equations were able to be obtained.

2.2.2 Architecture Limitations

While impressive in both practical scope and theoretical substance, this foundational paper is not without criticism, summarised in a critical analysis from Yuntian Hou et.al. [25]. Namely, while the parameter space scaling is quantified, there is no complexity metric in time provided. Given the significant complexity jump from linear weights to B-Splines, Hou et.al. were able to determine a computational jump of over one hundred times that of an equivalently sized MLP [25]. As previously discussed, the breaking of the COD by KANs is also in contention, and is a spurious claim given the function f used to determined the error bound must be representable by some infinite gridsize KAN. This is not an assumable condition for functions, as even if f is multivariate continuous, KART demonstrates that f can only be represented by the equivalent two-layer KAN of widths d and $2d + 1$ for input dimension d. This does not hold for a KAN with an arbitrary number of layers, thus the error bound is not a genuine result [25]. In fact, Rupeng Yu et.al. [72] demonstrate that, aside from interpretability, KANs underperform

MLPs under similar computational resource constraints and parameter counts. In the fields of machine learning, natural language processing, computer vision and audio processing, MLPs with GeLU or ReLU activation performed better than equivalently sized KANs given the same number of floating point operations (FLOPs) during training. Additionally, from a skeptical point of view, these datasets were potentially chosen to 'cherry pick' the papers results, as is becoming an increasingly common practice in academia [52]. Moreover, there are significant assumptions in the underlying functions, notably requiring that the inner functions of KART are smooth to derive the KAN approximation bound. Determining the smoothness of these inner functions is not a skill expected from the general user of a neural network, which inherently discourages general adoption. Even the authors themselves label their mathematical theory as *"limited"*, and the expensive computational costs *"more as an engineering problem to be improved in the future rather than a fundamental limitation"* [42].

2.2.3 Subsequent Developments

Despite these substantial concerns, there have been significant developments in KAN architectures since the publication of the seminal paper in 2024.

Computational complexity is a key component of practical adoption of machine learning architecture, thus the original B-Spline KANs were not feasible for widespread use due to expensive computations. Ziyao Li, ten days after the publishing of the original KAN paper, identified that the 3-order B-Splines used in the paper were well-approximated by gaussian radial basis functions, which provided a 333% speed gain to the network's forward pass, creating an algorithm, dubbed FastKAN [37]. Identifying the potential for adaptivity and generalisability to complex data in KANs, Zavareh Bozorgasl and Hao Chen proposed the use of wavelet functions as basis functions, creating Wav-KAN [11]. The two argue that B-Splines, used in the original architecture, provide good local control, having use cases where precision is the primary target, such as computer graphics. However, they perform poorly suited to high dimensional data due to increasing complexity and an inability to deal with sparsity. The use of wavelets instead better handles sparsity, and can more easily deal with noise in identify overarching data structures. While there are additional choices in the model, such as the type of wavelet functions used, Wav-KAN maintains the interpretability of B-Spline KANs whilst significantly improving model performance [11]. As of June 2026, this paper has surpassed the original KAN paper in the number of citations received, reiterating the importance of this development. Numerous other basis functions have been highlighted for viability in the KAN architecture, including Chebyshev polynomials [55], rational functions [3], Jacobi functions [2], Sinc functions [73], and combinations of B-Splines and radial basis functions [59]. Liangwewi Nathan Zheng et.al. demonstrated that the instability often seen in B-Spline KAN training was due to spline oscillations in fine grids, where large variations in second derivative values

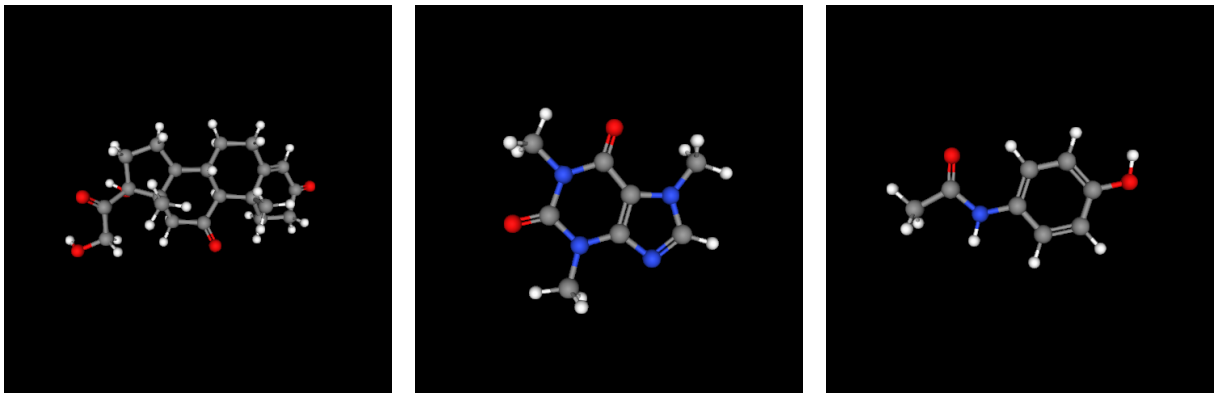
create unstable training processes [76]. By introducing a regularisation term for the second derivative of the splines, the training stability of the KAN can be improved. The initialisation of the KAN parameters has also been identified as a key choice for suitable model performance. Spyros Rigas et.al. demonstrated that for large numbers of parameters, B-Splines KANs exhibited significantly varied model performance depending on the initialisation technique, leading to the identification of a *"Glorot-inspired"* optimal initialisation scheme [49].

The most relevant of these developments for the purposes of this thesis is the creation of the FourierKAN, originally published to Git by a company Gist Noesis [20] and first used in a paper by Jingfeng Xu et.al. [68]. A natural consideration for function approximation, Fourier series exhibit certain properties that make them more appealing than B-Splines for use in a KAN. They capture periodic behaviours, and may better represent functions with identifying features in the frequency domain. Jusheng Zhang et.al. were able to implement Kolmogorov Arnold Fourier networks (KAF), using specifically Random Fourier Features, that produce State-of-the-Art performance in both computer vision and natural language processing tasks [75].

2.3 Molecular Embedding

The study of molecular interactions within human biological contexts is a primary field of research in modern molecular biology. Studies within this field take a wide range of forms, from toxicological profiling to identifying promising drug candidates to revealing the mechanistic molecular interactions that underlie a biological process. Developments within these fields have the potential to advance pharmacological and biomolecular understanding, which can in turn, positively impact the lives of millions of people. The evolving field of precision medicine, for example, seeks to integrate information from a person's molecular landscape, including their genetics, into individually tailored medical treatment or advice, a field that would not be possible without the study of molecular interactions [40]. The importance of this field, therefore, cannot be understated.

To study molecular interactions, properties of the molecules themselves must be well understood. A molecule, by definition, is composed of atoms that are bonded together. In the biological context, the term molecule is often used in place of 'organic molecule', which primarily consist of carbon, hydrogen, nitrogen and oxygen, often in complex structures. These molecules exist in 3D space, with structures called 'steric arrangements'. These structures are dictated by the physical interactions between individual atoms. Features such as static charge, atomic radius and atomic weight can push or pull nearby atoms, force certain three dimensional 'conformer' structures in the molecules, and determine the types and properties of bonds between atoms. Some molecular structures are shown in Figure 2.1. Just as these atoms dictate the internal structure of the molecule, so too



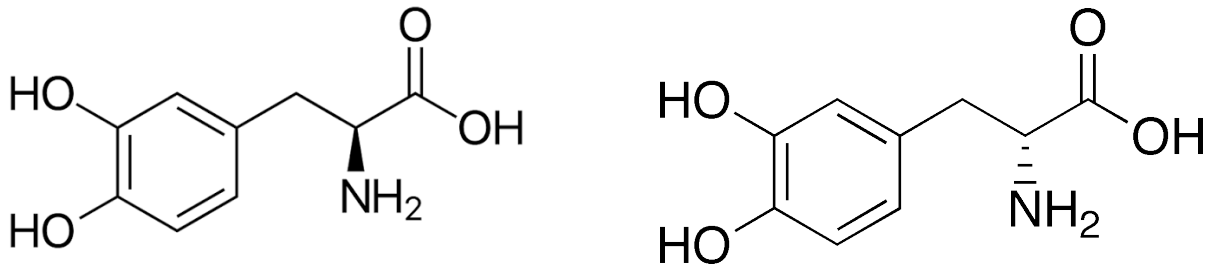
(a) Cortisone (b) Caffeine (c) Paracetamol

Figure 2.1: The 3D structures of a selection of molecules. Grey represents a carbon atom, blue is nitrogen, red is oxygen, and white is hydrogen

do they influence the interaction of the molecule with it's environment. Therefore, the interaction of a molecule with a specific target, such as a protein, a biological receptor, or a DNA strand, is entirely dictated by the arrangement and properties of atoms and their bonds that form the molecular structure.

One natural way to represent a molecule is through an undirected graph. The atoms are analogous to the nodes of a graph, with the bonds between atoms corresponding to edges. Whilst the 3D structure at first appears to be lost through this representation, the inclusion of feature vectors for both the atoms and bonds can essentially fill in this information. As mentioned, the 3D structure of the molecules is determined by it's constituent atoms and their arrangement, which can be is entirely preserved within the graph representation through atomic feature values associated with each node, and their corresponding topological arrangement. By including feature vectors for the bonds within the edges, the 3D structure of the molecule is more explicit, and can bypass the need for physical interaction models in determining steric arrangements.

A molecule contains an exceptional amount of complex detail. Aside from the individual, mainly fixed elemental properties of the atomic constituents, there are aspects such as bond hybridisation, interatomic distances, ring structures, electron orbital conjugations, chirality and stereoisomerism such as enantiomers, functional groups such as alcohols and aldehydes. The list in near inexhaustable. It is these finer details of molecules that can lead to massively different biological interactions. Consider, for example, the molecule L-DOPA. This molecule is a precursor to dopamine, an important neurotransmitter critical to the function of the human brain. L-DOPA, once administered to a patient, can cross the barrier between the bloodstream and the brain known as the Blood-Brain Barrier (BBB), where it is then catalysed into dopamine [48]. Dopamine itself cannot cross the BBB, so L-DOPA is an important drug for treating conditions involving reduced dopamine



(a) L-DOPA (b) D-DOPA

Figure 2.2: The stereochemical skeletal structure of L-DOPA and D-DOPA. Note the differing linestyle from the backbone to the nitrogen atom. These wedges mean that, in 3D space, the amino group in L-DOPA is pointing 'out of the page', but for D-DOPA it is 'into the page'.

levels [13]. D-DOPA is a near identical molecule to L-DOPA, being made of the exact same atoms in nearly the exact same structure. The only difference is that the two are mirror images of one another, known as enantiomers. There is a single group of atoms, the amino groups, that points in a different direction in each molecule, seen in Figure 2.2. This change, though small, make D-DOPA unable to pass through the BBB, thus rendering it useless in treating the same conditions as L-DOPA.

This example highlights the role that molecular intricacies play in chemical interactions, with even minute differences leading to drastically different behaviour. It is natural to conclude that, by understanding as much detail in a molecule as possible, the subsequent interactions it will have with various other molecules can be better understood and predicted. This is true, in a sense. Thalidomide, for example, once used as an anti-nausea drug for pregnant mothers leading to severe birth defects among children [61], was later revealed to restrict blood vessel growth once it's chemical structure was well understood [10]. Now, through the elucidation of it's molecular structure, this historically tragic drug is used to treat blood and bone diseases such as Multiple Myeloma[8]. Moreover, there also exist alerts for molecules with recorded structure that possesses atomic groups or arrangements with known toxic effects [38], even if the compound itself has not been chemically synthesised before.

Greater knowledge of molecular structures has quite clearly led to improved understanding of molecular interactions in complex biological systems. Yet the hopes to exactly understand every single interaction possible in such systems is little more than a pipe dream. Proteins, the massive molecules key to every single biological process, individually possess thousands of individual atoms, in structures of overwhelming complexity [4]. Only 17% of all known proteins have experimentally confirmed structures, and even with advancements in AI prediction techniques such as AlphaFold, there is only confidence in about 36% of them [62].

While a drug molecule can have it's structure and constituents determined relatively easily in the present day [60], the variety of biological molecules like proteins, RNA and DNA do not share this same ease, making it difficult to elucidate exact interaction mechanisms and, in turn, physiological effect. It is for this reason that, rather than investigating exact interactions on a case-by-case basis, machine learning techniques are often implemented in prediction interaction outcomes. Given the 'black box' nature of most biological interactions, machine learning is a natural fit for such tasks, offering reasonable predictive performance even when the underlying mechanisms that link the input and output are not well known theoretically.

2.4 Graph Neural Networks

In situations such as screening for potential drugs to treat a specified disease, it is significantly more efficient for molecules to be screened on a large scale for potential effects of interest, such as in massive multi-parallel screening regimes [30]. To do this, a molecular embedding mechanism can be used. By creating an embedded space that preserves all of the properties of a given molecules, the computational complexity in dealing with graph-structured molecules can be circumvented. An example of a basic molecular embedding is the Simplified Molecular Input Line Entry System, better known as SMILES. SMILES representations take 2D skeletal molecular representations and convert them to character strings that can then be interpreted by computers. While SMILES representations themselves are non-unique, and do no account for important 3D molecular properties such as bonding angles, they are popular in molecular modelling due to their ease of acquisition and string-based format that lends naturally to Natural Language Processing (NLP) models [46].

Given the inherently graphical representation of molecular structures, Graph Neural Networks (GNNs) are often used for molecular embedding and prediction tasks [9]. Molecular graphs can be encoded with as much necessary detail as required within their node and edge feature vectors, which are then sent through message passing layers that aggregate neighbourhood information into each node, followed by a readout layer that can produce an embedded version of the input molecule, or a direct prediction for a given use case. Neighbourhood information is often critical to the role that individual atoms in a molecule play in certain interactions, meaning the architecture of a GNN captures the physical understanding of the individual importance of atoms. GNNs come in three main groups: Graph Convolutional Networks (GCNs) use message passing layers to convolves established node neighbourhoods into updated node features, and are the most basic kind of GNN. Graph Attention Networks (GATs) use the same concept of a GCN but apply attention coefficients to prioritise relevant information transfer during message passing layers. Graph Autoencoders (GAEs) are GCN or GAT architectures applied within an

autoencoder, with a GNN encoder mapping to a latent space, which is then fed into a decoder to reconstruct either the full graph or some desired embedded properties of it.

GNNs are naturally permutation invariant. This means, for a given ordering of the nodes, as long as the edges are consistent between nodes, the output will be identical. Such a feature is critical for graph-structured data, as typical topological representations such as adjacency matrices have $N!$ equivalent representations, where N is the number of nodes in the graph. Message passing layers allow the information of a node's neighbourhoods to be incorporated into the node values, allowing an incorporation of structural information into the values fed into each network layer. This incorporation of topology means that GNNs can capture more information on a graph-representable object than a standard neural network taking just the node and edge features. Moreover, GCNs and GATs do not require a fixed input size, and do not have restrictions on the graph structures that an object can take, including disconnected graphs. These features make them optimal choices for machine learning tasks where data may contain varied topological structures or sizes, such as in molecular datasets or social networks. The popularity of GNNs in recent times can be summarised in the response to the 2020 paper *"Graph neural networks: A review of methods and applications"* by Jie Zhou et.al., which, as of July 2026, has received over 10,000 citations [77]. For molecular property prediction in particular, GCN, GAT and GAE architectures dominate the landscape of existing models.

Chapter 3

Motivation

3.1 KANs in GCNs and GATs

Following the publication of the seminal KAN paper [42] in 2024, it was not long until applications began to spill into GNNs and molecular embedding. In August of 2025, Longlong Li et.al. published the paper *"Kolmogorov–Arnold graph neural networks for molecular property prediction"* [36]. This paper bridged the gap between KANs and the popular GCN and GAT architectures used in molecular embedding models. By applying B-spline, polynomial and Fourier basis KANs in place of the typical MLP modules in the GNN layers, it was posited that the KAN equivalents to both GCNs and GATs exhibited greater performance across a range of molecular prediction tasks. Notably, the performance gains were shown to be greatest when using a Fourier-based approach across all datasets, with the B-spline implementation exhibiting poorer performance than the standard MLP-based GNNs. The reported performance of the KA-GCN and KA-GAT models was so great, in fact, that they were reported as having the highest known performance across a variety of molecular embedding benchmark datasets, making them State of the Art (SOTA) models.

This paper forms the basis for much of the architecture of this thesis, and although peer reviewed and published in a Nature journal, it was noted during research that there are potential issues in the soundness of the reported results. Namely, no raw data of the experiments conducted has been made available, and significant discrepancies exist between the reported methods and the actual code used. The paper, initially published in August 2025, has had significant changes made to the graph pre-processing code posted on the project's Github [43]. The initial feature generation of the molecular graphs was almost nonsensical, such as significant conditions filtering for the opposite logic of what was intended. Since February 2026, there have been numerous changes and fixes to these processes from the authors. Nonetheless, it remains unclear how the results of the paper were generated, as the procedure in not reproducible. Aspects of the code, once edited,

did replicate the methods outlined in the paper, however a full reproduction was outside the scope of this thesis.

On the basis of being published in a peer-reviewed journal, the paper by Li et.al., along with its SOTA performance metrics, will be taken as truth, throughout this thesis. However, the author notes that the scrutiny of the reported results remains necessary in preserving the objectivity of any conclusions drawn against the paper's reported results.

3.2 Importance of GAEs

Notably, this paper by Li et.al. leaves an open gap in research. While KAN modules were shown to provide an advantage in molecular prediction tasks in GCN and GAT architectures, the authors did not attempt to apply such modules to the GAE class of architectures. GAEs are a particularly interesting subset of GNNs. Their autoencoder structure allows the compression of graph objects into a lower dimensional latent space, essentially learning to compress key features of the graph into a representation that is much easier to deal with than the initial graph structure. What this means in the context of molecular embedding is that a single GAE could be trained on a large context window of molecules, and rather than using the entire graph for a prediction tasks, the pre-trained embedding vector can be fed into a small prediction module. This single GAE being used in all downstream prediction tasks massively reduces the size of prediction architectures required, requiring only the initial training of the GAE, and subsequent prediction modules that would be orders of magnitude smaller than equivalent GCN or GAT modules. The ability to richly embed complex molecular graphs with massive arrays of features into a low-dimensional latent space means that prediction algorithms for molecular interactions can be operated with massively reduced computational resource requirements while maintaining the predictive performance of a larger, graph-input architecture. This is exhibited diagrammatically in Figure 3.1.

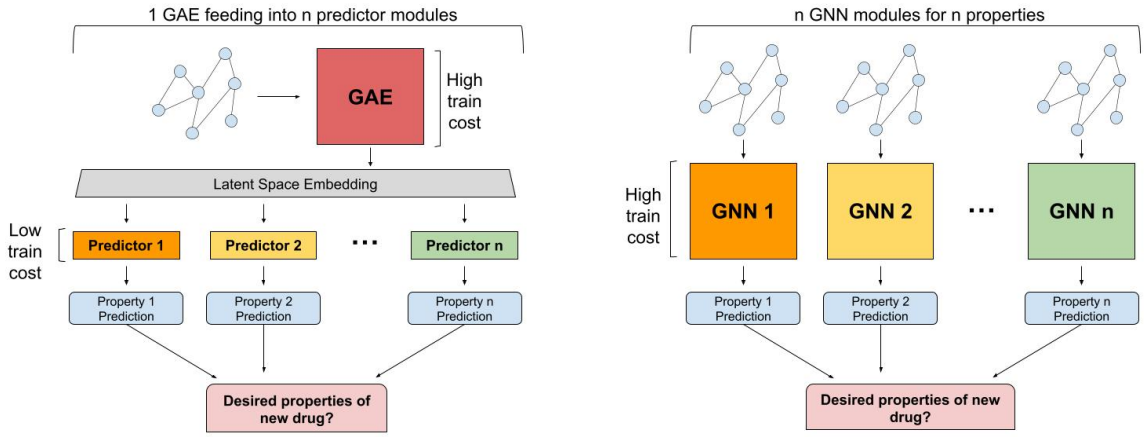


Figure 3.1: A comparison of a hypothetical drug screening pipeline for a given molecule, using both a GAE (left) and other GNNs (right). Using the GAE, there is only one significant resource cost in the training of the GAE, with subsequent prediction modules for properties being much smaller in comparison. On the other hand, using traditional GNNs for each property prediction is more computationally draining, requiring the training of each GNN module.

If there are n properties of interest during drug design, the amount of resources required to create a screening pipeline is dominated by the initial cost to train the GAE, as the subsequent prediction modules on the latent output are lightweight in comparison. In contrast, if GNNs are used for each property prediction, the computational cost scales with n, which is often large as there are many conditions required for a drug to be deemed suitable for further testing [28]. Even if a single GNN was trained for all property prediction, it would require a sacrifice of either individual predictive performance across properties or a massive increase in size that would counteract any gains compared to using individual modules

In practice, an effective GAE facilitates massive productivity gains in pharmacological research, namely in the identification of novel drug targets for the treatment of disease. By reducing the computational requirements for the screening possible interaction effects, both desired and undesired, the first stage of drug development can be conducted significantly faster, especially in combination with massively parallel processing and high-throughput screening techniques.The impact of a GAE with a richer embedding space on the identification and development of new drugs, therefore, cannot be understated. The application of KANs to GCN and GAT architectures has demonstrated gains in predictive performance.

3.3 Motivation and Intent

In the current research landscape, there are no known applications of KAN modules to GAEs in any research domain. There exists a clear precedent for performance enhance-

ment of other GNN architectures, namely GCN and GAT, through the addition of KAN modules in the context of molecular property prediction.

This thesis aims to bridge the literature in the application of KANs to GAEs through the creation of a novel machine learning architecture. Of practical significance, this study seeks to determine if a KAN-based GAE can produce an enhanced molecular embedding space for greater downstream predictive performance than a conventional MLP-based GAE.

Chapter 4

Models

4.1 Contrastive Kolmogorov-Arnold Network Graph Autoencoder

The Contrastive Kolmogorov-Arnold Network Graph Autoencoder (CKANGA) follows the general architecture of a Graph Autoencoder (GAE) with standard MLP modules replaced with Fourier KAN modules.

The Fourier KAN module, for node $x_j^{(l)}$ in layer l with width n_l and K harmonics, is defined as:

$$x_j^{(l+1)} = \sum_{i=1}^{n_l} \sum_{k=1}^{K} A_{ijk}^{(l)} \sin kx_i^{(l)} + B_{ijk}^{(l)} \cos kx_i^{(l)} \tag{4.1.1}$$

The encoder section consists of a KAN-based Graph Convolutional Network (GCN). Denoting the feature vector of node v as n_v, and the feature vector of the edge connecting nodes v and u as $e_{v,u}$, the encoder takes an arbitrarily sized graph g and runs the following processes:

1. **Node Embedding** - Node vectors are updated to include aggregate information of their neighbourhood edge vectors through concatenation with the mean of all neighboring edge vectors. This is then run through a single layer KAN. Mathematically, with output of a single KAN layer given input x denoted as $KAN(x)$, and the neighbourhood of node v denoted as $N(v)$, node feature vectors are updated as:

$$h_v^{(0)} = KAN\left(concat\left(n_v, \frac{1}{|N(v)|}\sum_{u \in N(v)} e_{v,u}\right)\right) \tag{4.1.2}$$

2. **Message Passing** - The embedded node values are then processed using a single layer KAN, which is then passed to neighbouring nodes with degree normalisation to prevent dominance of highly connected nodes. This process is then repeated iteratively, with each message passing layer providing a wider 'view' of the graph to each updated node value. Assuming node v is not in it's own neighbourhood, and

with the degree of node v given as d_v, at the k^{th} message passing layer, the updated node vector at node v is given as:

$$h_v^{(k)} = \sum_{u \in N(v) \cup \{v\}} \frac{1}{\sqrt{d_v}\sqrt{d_u}} \cdot KAN\left(h_u^{(k-1)}\right) \tag{4.1.3}$$

3. **Readout** - The processed node features for the entire graph are then aggregated using mean pooling. This produces a single vector value, which is then processed using a multilayer KAN to produce the latent embedding of the autoencoder. Mathematically, with L_g as the latent output of graph g with size $|g|$, after M message passing layers, and with r readout layers, this can be written as:

$$L_g = KAN_r \circ KAN_{r-1} \circ \cdots \circ KAN_1 \left(\frac{1}{|g|} \sum_{v \in g} h_v^{(M)} \right) \tag{4.1.4}$$

The decoder module takes the latent space vector output of the encoder module and processes it through a multilayer KAN. When training, the decoder attempts to reconstruct two things:

- **Graph-Level Features** - The autoencoder attempts to encode information about the graph's node values into the latent space. This is done through an L1 (or MAE) loss term that measures the reconstruction of the sum of all node feature vectors. Denoting this feature loss as l_f, and the decoder feature output as $\boldsymbol{y^{(f)}}$, with node feature dimension D, the loss is formulated as:

$$l_f = \frac{1}{D} \sum_{i=1}^{D} \left| S_i - y_i^{(f)} \right| \tag{4.1.5}$$

  With $\boldsymbol{S} = \sum_{v \in g} n_v$
  This error term is irrespective of the graph topology, acting to encode only the feature information of the graph.

- **Graph Topology** - The autoencoder also attempts to encode the topology of the graph into the latent space, ensuring information on graph structure is present. To do this, an L1 loss term is introduced that encourages the decoder to reconstruct the 10 smallest, non-trivial eigenvalues of the graph Laplacian. The Laplacian is a matrix representation of the topology of a graph, with the eigenvalues of this matrix forming the spectrum of the graph. Even though, for a graph with D nodes, there are $D!$ equivalent Laplacians, the spectrum of a graph is permutation invariant, meaning that all representations of the same graph have the spectrum. The ordered spectrum of a graph captures key topological features of the graph, such as the Algebraic

Connectivity that measures the ease of partitioning. Assuming the input graph is connected, there will always be a Laplacian eigenvalue of 0. The topological loss term then tries to reconstruct the next 10 smallest eigenvalues, ensuring that structural information of the graph is included within the latent space. Mathematically, with the 10 smallest eigenvalues of the graph Laplacian given in vector $\boldsymbol{\lambda}$, and the decoder feature output as $\boldsymbol{y^{(T)}}$, the topological loss term l_T is given as:

$$l_T = \frac{1}{10} \sum_{i=1}^{10} \left| \lambda_i - y_i^{(T)} \right| \tag{4.1.6}$$

On top of these reconstruction loss terms, a batch-wise contrastive term is also included. This contrastive aspect promotes a higher quality latent space by ensuring similar graphs are embedded closer together than dissimilar graphs. Without this term, the latent space may lack separability and poorly embed unseen input graphs, which is important for higher performing downstream predictive modules. For the molecular graph data, the contrastive term utilises a batch-wise Tanimoto similarity score, calculated on a binary molecular embedding vector known as a Morgan fingerprint. For Morgan fingerprint F of length B, with $n = \sum_{i=1}^{B} F_i$ and the intersection between two fingerprints given as $D_{a,b}$, the Tanimoto similarity is given as:

$$T_{a,b} = \frac{D_{a,b}}{n_a + n_b - D_{a,b}} \in [0, 1] \tag{4.1.7}$$

The contrastive loss term then uses this similarity score as a scale for a similarity or dissimilarity penalty. Given the latent embedding of two graphs, with the euclidean distance between them as $d_{a,b}$, the dissimilarity margin as m, and a non-diagonal mask matrix $M = J - I$ used to remove self-similarity terms, the contrastive loss l_C is given as:

$$l_C = \frac{\sum_{i=1}^{B} \sum_{j=1}^{B} M_{i,j} \left(T_{i,j} \times d_{i,j}^2 + (1 - T_{i,j}) \times (\max(0, m - d_{i,j}))^2 \right)}{\sum_{i=1}^{B} \sum_{j=1}^{B} M_{i,j}} \tag{4.1.8}$$

Importantly, while the similarity term $T_{i,j} \times d_{i,j}^2$ pulls all embeddings with any similarity closer together, the dissimilarity term $(1 - T_{i,j}) \times (\max(0, m - d_{i,j}))^2$ only pushes apart embeddings within the specified margin distance, encouraging the space of embeddings to remain more 'compact'. This denser latent space aims to better associated distances between embeddings with semantic meanings of input graphs, in turn improving the generalisation of the autoencoder to unseen input graphs once trained. The overall loss function l of the autoencoder, with topological weighting t and contrastive weighting c, is therefore given as:

$$l = (1 - t) \times l_f + t \times l_T + c \times l_C \tag{4.1.9}$$

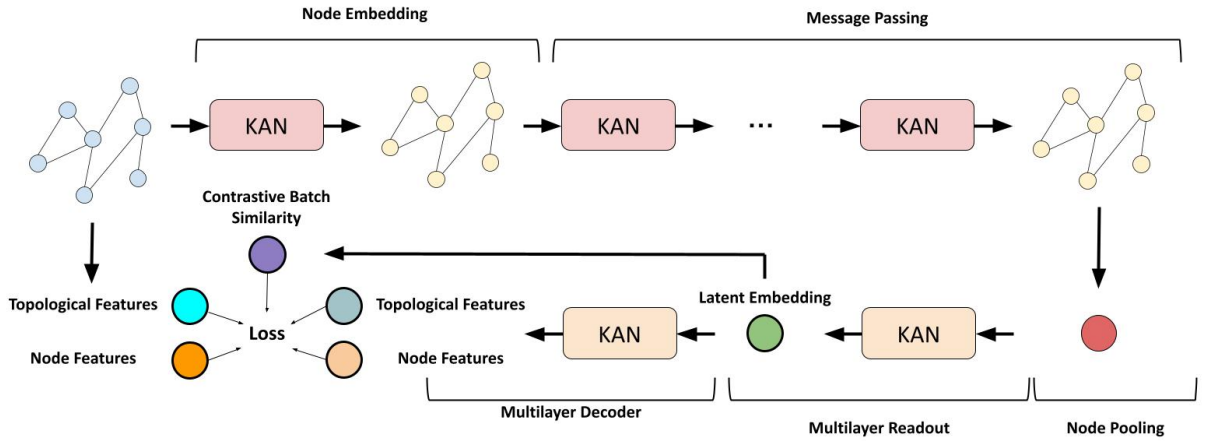


Figure 4.1: A simplified representation of the CKANGA architecture. Graphs are first passed through a node embedding KAN, followed by multiple message passing KANs that update node values. These nodes are then aggregated into a single vector, which is passed through a multilayer readout KAN to yield the latent embedding. The decoder, consisting of a multilayer KAN, attempts to reconstruct both topological and graph-level node features, whilst minimsing the contrastive loss penalty to ensure latent space separability.

A diagrammatic representation of the CKANGA architecture can be seen in Figure 4.1.

4.2 Data Processing and Setup

To prevent excessive computational resource requirements, a custom assignment of molecular node and edge features was created. This feature generation program was inspired by Li et.al. [36], leveraging the author's chemistry domain knowledge to produce a reduced feature size of 23-dimensions for nodes and 10-dimensions for edges, totalling a 33-dimensional input after node embedding in comparison to the 100+ of Li et.al.. A breakdown of the generated features can be found in Appendix 8.1.

To ensure that performance could be achieved across the literature, datasets from the MoleculeNet benchmarking database [67] were selected for use in testing and tuning. The 'BACE' dataset was selected for hyperparameter tuning given it's small 1700 entry size, while the 'HIV', 'BBBP' and 'Tox21' sets were chosen for testing. All of these dataset involve classification tasks, with all but Tox21 being binary classification. Descriptions of these datasets can be found in Appendix 8.2. Results from testing on the BACE dataset were included in reporting for completeness.

An important detail pertaining to these MoleculeNet sets is the method of splitting. In chemical contexts, models are often created with the goal of generalising to unseen structural sets rather than just unseen individual molecules. By doing this, performance metrics are seen to be more representative of the use case context of the model, which

is often some form of drug discovery [16]. This method is called Scaffold splitting, and creates a non-random split where the test set consists primarily of molecules with structural components that are not present in the training set, testing the model's ability to generalise across 'novel' chemistries [67]. Scaffold splitting leads to generally reduced model performance metrics due to the unseen structures, however the deterministic nature means that it is useful for benchmarks like MoleculeNet. The datasets used here, except Tox21, follow MoleculeNet's reccomended scadffold splits of 80/10/10 to ensure conclusion are drawn on the same data splits as the wider literature. Scaffold split itself is only really qualitatively justified [22], but provides an interesting alternative to random splitting that promotes fairer performance comparisons.

To provide a sufficiently large context window for CKANGA, a selection of 200 thousand, 500 thousand and 1 million molecules from the ChEMBL Small Molecules Database were used in the training of the autoencoder [74]. This selection process involved filtering of molecules with at least eleven atoms to ensure that 10 non-trivial laplacian eigenvalues could be derived, followed by random shuffling and selection. The selections were made such that the larger sets contained the entirety of the smallers sets (eg. the 200 thousand molecule set was contained in the 500 thousand set, which was itself in the 1 million set). To investigate the effect of a varied context window, each subset was each used to train a separate CKANGA model. These sets were processed using the same molecular feature as those derived for the test sets.

As a result of the custom feature engineering used in this thesis, fair comparisons of the CKANGA architecture against existing models in literature are difficult to make. The features derived are significantly fewer than those used in other studies, and thus the model may perform inherently worse as a result of less input information. In order to ensure a fairer comparison for CKANGA, a model replicating the CKANGA architecture but with MLP layers in place of KAN layers was also created, which will be referred to as 'CMLPGA' or just 'MLP'. Likewise, the CKANGA model may also be referred to as 'KAN'.

All data processing, tuning and model setup was conducted on the University of New South Wales Katana shared computational cluster [63].

4.3 Parameter Tuning and Training

To more optimally tune the performance of the CKANGA and CMLPGA architectures, a sequential tuning approach was adopted rather than a grid search. Initially, a small grid search was attempted, however, the runtime of the tuning program exceeded a twelve hour walltime, and had only covered a minimal section of the grid, demonstrating that a less computationally intensive approach was required. The CKANGA model contains 10 hyperparameters that must be tuned, with some significantly influencing the training

speed, such as the number of harmonics or hidden layer width. In light of computational constraint,s, the sequential approach taken assumed a base model state, and would tune a single parameter over a given range of values. The parameter with the greatest mean AUC was then added into the base model state, and the process repeated until all parameters had been exhausted. The ordering of the parameters was chosen qualitatively by the author, with the more 'important' features such as hidden width and message layers coming first, and secondary features such as the contrastive parameters at the end.

An 80/20 scaffold split of the BACE dataset was used for this tuning process. The BACE dataset is relatively small (1700), and while this limits the numbers molecules that the model is exposed to, it best suited the limited computational resources on hand. Tuning iterations consisted of 500 epochs of autoencoder training, followed by 500 epochs of training of a prediction module on th latent space, with the encoder frozen. An early stopping criteria was employed to further reduce runtimes. Even with these choices that undoubtedly restricted the efficacy of the tuning process, overall tuning took in excess of 30 hours for each CKANGA and CMLPGA. The results of this tuning procedure are outlined in Appendix 8.4.

Three autoencoders with the resulting tuned parameters were then trained on each of the CKANGA and CMLPGA architecture, with one for each size subset of the ChEMBL database (200k, 500k and 1M). Each autoencoder was trained for 500 epochs, after which, the test data was run through them. A small MLP prediction module was then and trained on the autoencoder's latent space output of training set, after being tuned over a small grid search on the validation set. The best AUC output over 500 epochs, with the same early stopping as in the tuning phase, was then recorded for 100 iterations on random seeds.

Chapter 5

Results

5.1 Initial testing

Following this procedure, the performance of both the CKANGA and CMLPGA on the BACE, BBBP, HIV and Tox21 datasets had been determined, summarised in Table 5.1.

While the CMLPGA model exhibited AUC values indicative of a learning model, the CKANGA architecture did not have a single model exceed an AUC of 0.6, with the 500k context BACE performance being 0.48. Initally, this was thought to be a issue with the

Table 5.1: Model results across MoleculeNet datasets (Best AUC-ROC value during testing epochs, over $n = 100$ seeds).

Dataset	**Mean**	**#AUC<0.5**	n
200K			
BACE	0.5564	0	100
BBBP	0.5242	20	100
HIV	0.5476	2	100
TOX21	0.5884	0	100
500K			
BACE	0.4876	68	100
BBBP	0.5160	43	100
HIV	0.5241	9	100
TOX21	0.5926	0	100
1M			
BACE	0.5401	23	100
BBBP	0.5269	28	100
HIV	0.5316	1	100
TOX21	0.5945	0	100

(a) CKANGA

Dataset	**Mean**	**% with AUC<0.5**	
200K			
BACE	0.7281	0	100
BBBP	0.6328	0	100
HIV	0.6748	0	100
TOX21	0.7221	0	100
500K			
BACE	0.7222	0	100
BBBP	0.7399	0	100
HIV	0.6450	0	100
TOX21	0.7200	0	100
1M			
BACE	0.7151	0	100
BBBP	0.7304	0	100
HIV	0.7091	0	100
TOX21	0.7173	0	100

(b) CMLPGA

programming behind the architecture or a bug in the testing script, however repeating the results pipeline while training the autoencoder on the BACE dataset only yielded the same high AUC of 0.8 that had been achieved during the tuning phase. The reproducibility of the tuning results lead to the conclusion that the CKANGA model was actually being trained and tested properly, and that the exceedingly poor performance was not an issue of the procedure, but rather the architecture.

5.2 Subsequent Models

Following these results, differences between the two architectures' hyperparameter choices were analysed. It was thought that, while necessary for practical purposes, the use of the small BACE set for tuning had potentially lead to parameter choices that generalised poorly to the wider chemical contexts that the autoencoders were trained on. For example, both the latent size and hidden width of CKANGA were four times as large as in CMLPGA. This massive complexity difference was thought to be the cause of the poor performance of CKANGA, with the tuning process leading to parameter choices that were 'overfit' to the BACE dataset. To test this theory, another model, labelled as 'KAN modified' was created with all of the same parameter values of the original CKANGA, except with the latent size and hidden width set to match those of the CMLPGA model.

5.3 Final results

Subsequent testing indicated positive performance gains from this reduced model, leading to a hypothesis that reduced KAN-based model complexity could lead to improved performance. To more thoroughly test this idea, a significantly more radical model was proposed, dubbed 'KAN mini'. This model would be the most reduced KAN-based GAE available, with no contrastive weighting, a fixed 50/50 topology weighting, a single message passing, readout and decoder layer, a hidden with of 16 and, a 32-dimensional latent space, and just one harmonic. This model, more of a KANGA now that the contrastive approach was removed, was then subject to the same training and testing regime as the other two KAN-based models.

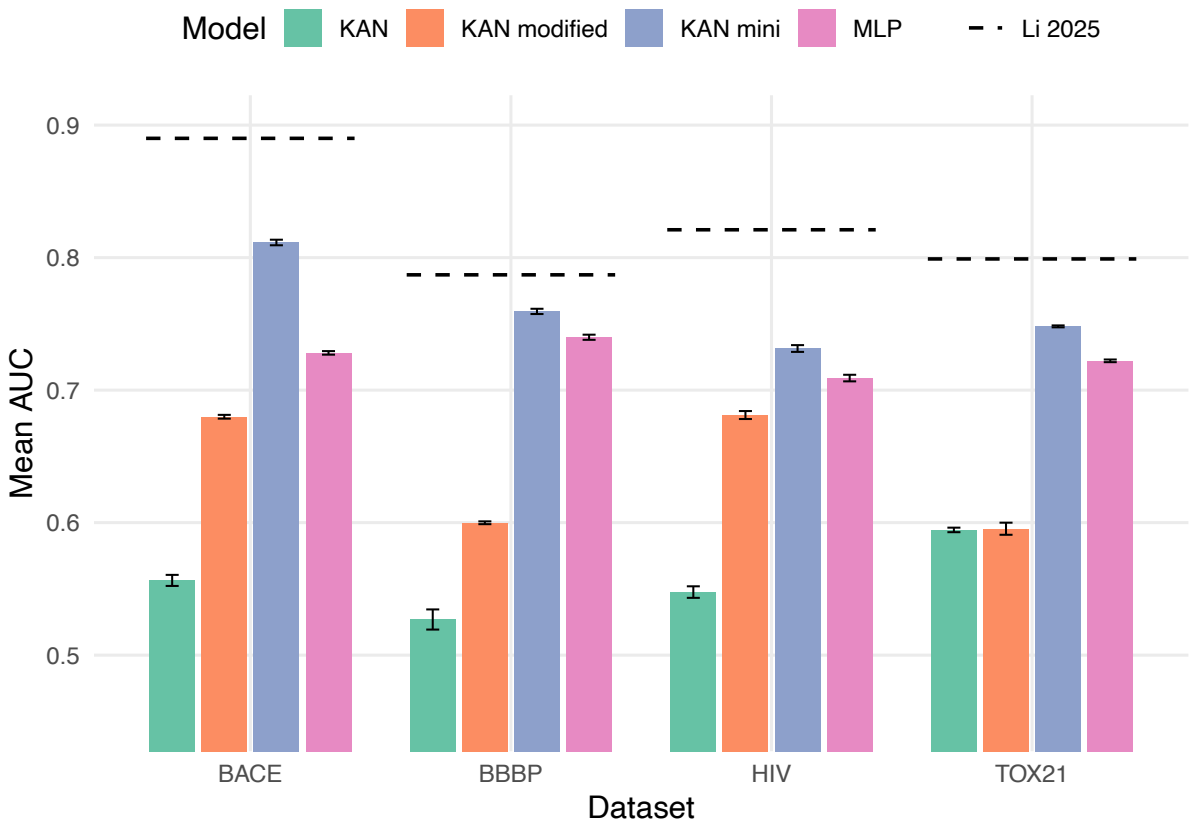


Figure 5.1: A clustered column chart comparing the best AUC performance of each model class for each dataset across all context window sizes. For comparison, State of the Art performance values of Li et.al. [36] are also given.

The comparative performance of each model class is shown in Figure 5.1 alongside the State of the Art (SOTA) AUC values for each dataset, as given by Li et.al. [36]. Only the best performing context size for each model is shown. The original CKANGA (labelled as 'KAN') performs much worse than all other models aside from in Tox21, where it is on par with 'KAN modified'. In BACE and HIV, the 'KAN modified' model class performs only slightly worse that the CMLPGA (labelled 'MLP'), while in BBBP the class performs between the CKANGA and CMLPGA models. The 'KAN mini' model class, in contrast, exceeds the performance of all other model classes, even CMLPGA, across all datasets, with the greatest performance gap seen in the BACE dataset.

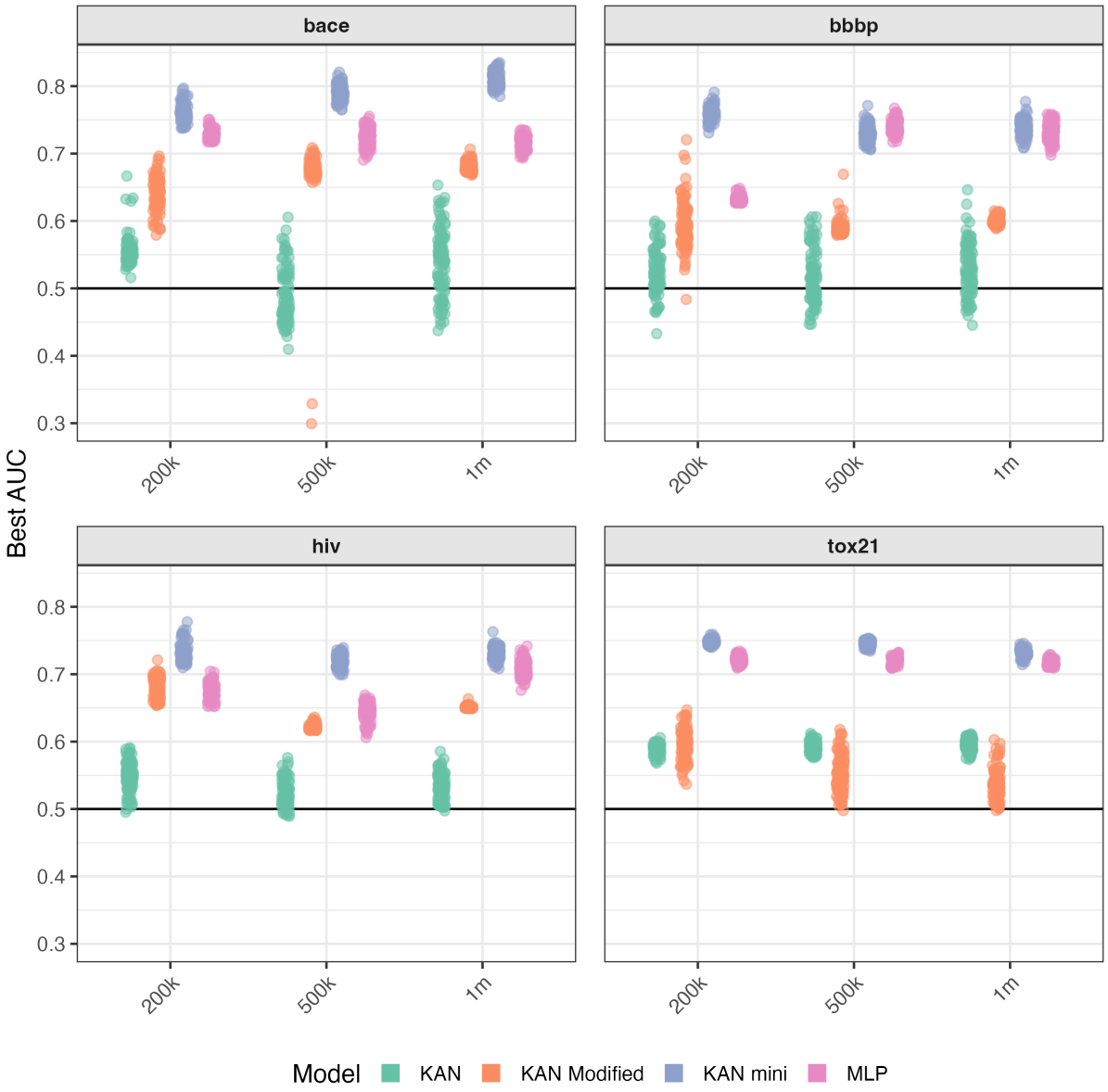


Figure 5.2: A jitter plot of the best AUC values for each model, context size and test dataset across 100 seeds. Note higher variance of 'KAN' and 'KAN modified' across all context sizes and datasets in comparison to 'KAN mini' and 'MLP'. 'KAN mini' is superior across all datasets and context sizes aside from 500k BBBP, where it is slightly below 'MLP'. 'KAN' exhibits the worst performance across all datasets except Tox21, where 'KAN modified' is worse at 500k and 1m. Note the two very low AUC values for 'KAN modified' in 500k BACE.

The best AUC values on each dataset across 100 seeds for each model and context size are seen in Figure 5.2. The plotted results reveal a significant volatility in both 'KAN' in nearly all occurrences except Tox21, and 'KAN modified' in the 200k context sizes and Tox21. 'KAN mini' and 'MLP' both have lower variance in most circumstances. 'KAN mini' demonstrates the highest average AUC score across all context sizes and test datasets aside from BBBP on 500k, where 'MLP' slightly beats it. In all datasets aside from Tox21,

'KAN' has multiple occurrences of best AUC values less that 0.5. 'KAN modified' also has a few occurrences of this in Tox21 and BBBP, but most strikingly in BACE 500k, where there are 2 occurrences only slightly above 0.3. Models do not generally appear to perform better across increasing context sizes, with varied trajectories across model classes.

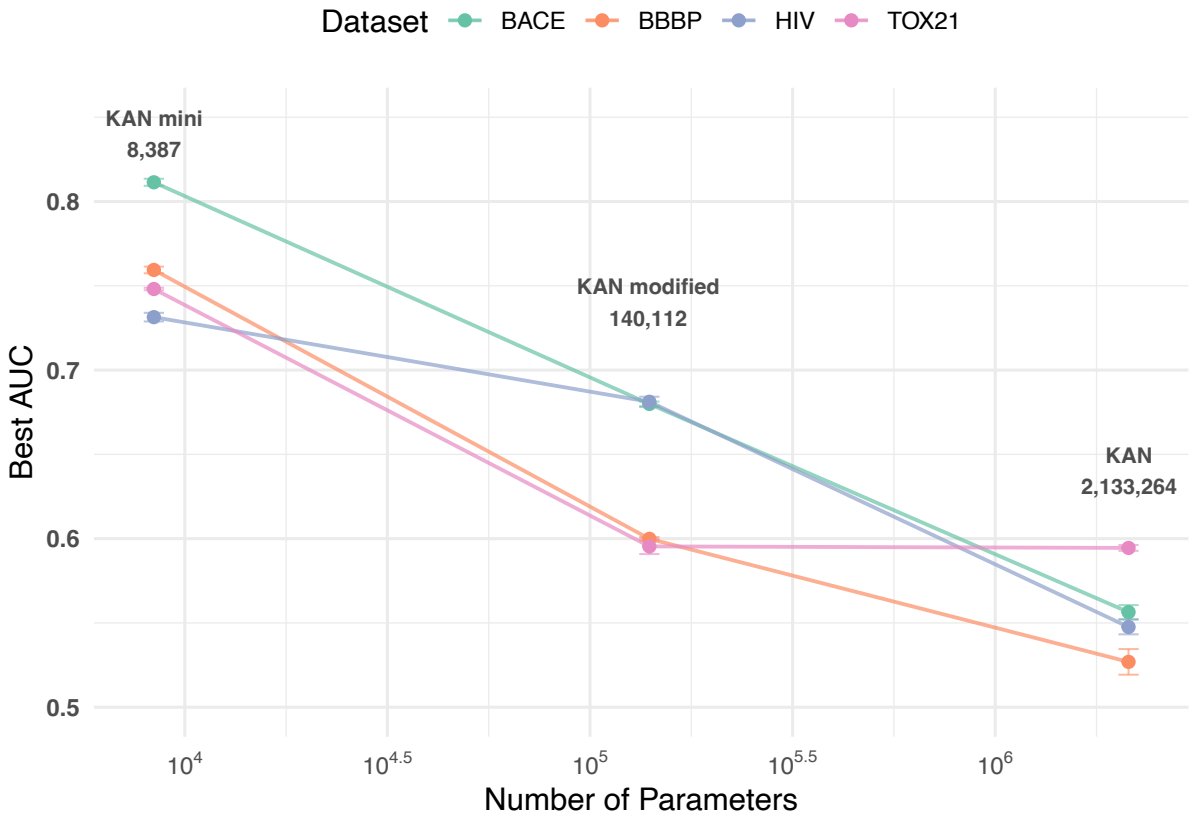


Figure 5.3: A line chart comparing model performance against the log of the parameter count for the KAN-based models. The results for dataset are given as a series.

A plot of parameter count of the KAN model classes against AUC is shown in Figure 5.3. Accuracy decreases as the number of parameters increases as a general trend present across datasets. For Tox21, there is no further reduction in performance when the number of parameters increases from 'KAN modified' to 'KAN'.

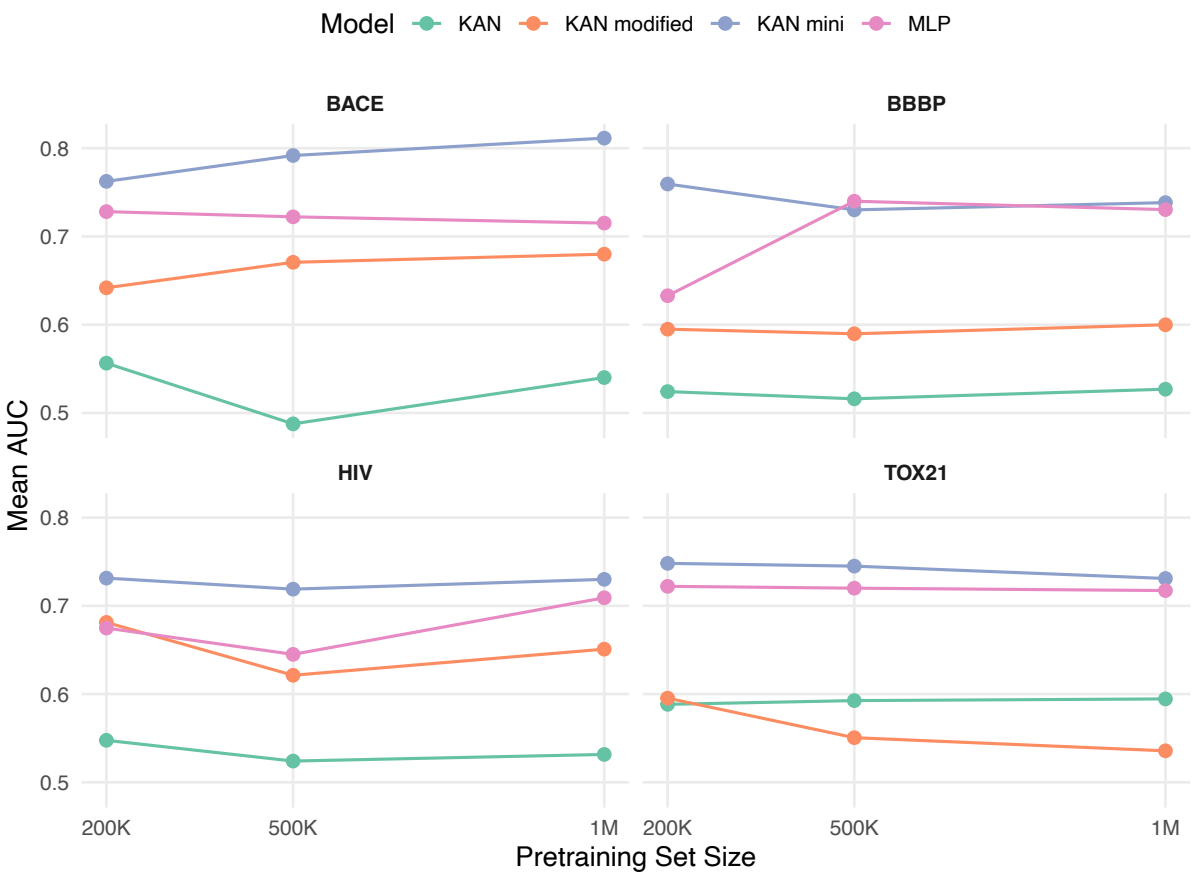


Figure 5.4: Line charts comparing model performance against the size of the context window for each model class on each dataset.

Plots of AUC against training dataset size for each dataset on each model class are shown in Figure 5.4. As the size of the training data increases, the model performance on each dataset remains relatively constant. There are minor variations across each dataset, such as the slight reduction moving from 200k to 500k in HIV, or the increased performance for MLP across the same gap. The 'KAN mini' model class was generally the more consistent across the context sizes than all other model classes.

Chapter 6

Discussion

The results obtained within this thesis are exceptionally intriguing, namely in how divergent they are to traditional schools of thought surrounding MLP-based machine learning. The demonstration of convention defying behaviour of KAN based networks further reveals the limitations, strengths, and possible use cases of the architecture, not just in molecular embedding but in the wider context of practical network architectures.

The most striking result is undoubtedly the relative performance of the KAN, KAN modified, and KAN mini model classes. The concept of double-descent has become a well known feature of machine learning algorithms in recent years, positing that, after an initial performance degradation due to overfitting, model errors again decrease as more and more complexity and depth are added to them [47]. Scaling a model's size, at least in traditional machine learning algorithms, should lead to some increase in performance, even if gains are not strictly linear. The CKANGA models do not obey this convention. From the results, the massively complex original CKANGA model class, the largest of the KAN-based model classes with over two million parameters, was comparable or worse than random guessing on some datasets. When the layer width and latent space were reduced to create the 'KAN modified' class, performance on all datasets aside from Tox21 signficantly improved. Going even further, the smallest possible KAN-based GAE in 'KAN mini', with nearly all the complexity of the original CKANGA class stripped out, outperformed the CMLPGA model to consistently output the best performance across all datasets, getting quite close to SOTA performance on the BBBP dataset. This result is very significant for KAN-based architectures, highlighting the potential for an inverse relationship between model complexity and performance that exactly the opposite of traditional understandings of machine learning algorithms.

This is not a definite conclusion, however. There are many factors that could be causing these results, and some may be specific only to the context of KAN-based GAEs or certain aspects of the models used. Possible causes are outlined in the following sections.

6.1 Unstable training dynamics at high complexity

B-spline based KANs are understood to have unstable training dynamics in certain experiments [56] [57]. While the fourier-based model of Zhang et.al. did not demonstrate such instability, there is the potential for the training dynamics to have broken down in the most complex KAN model. After training, the KAN model classes had their parameters investigated for signs of training difficulties. It was found that the parameters of a layer, either readout or message passsing, of the encoder in both 'KAN' and 'KAN modified' were consistently ten times smaller than those of the adjacent layers, a problem in all the models regardless of context size. A visual representation of this parameter size inconsistency is shown in Figure 6.1

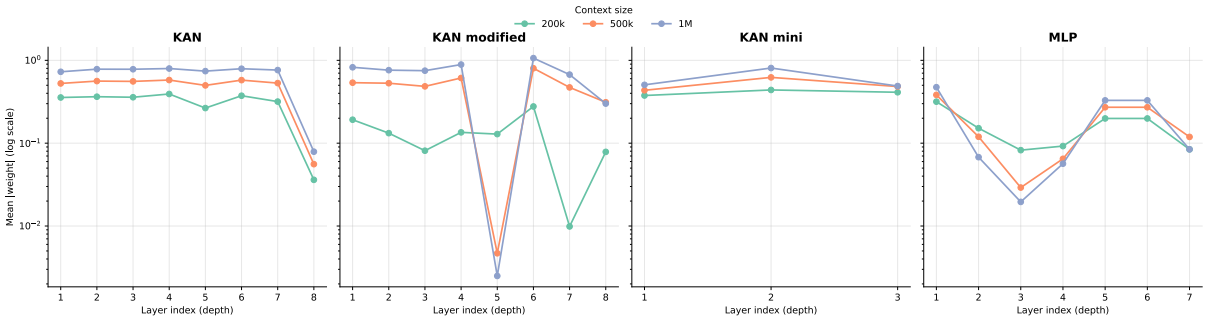


Figure 6.1: Comparisons of average model parameter size across layers for each model class on each context size. Note the sharp drop in layer 8 for KAN, and the drops at layers 5 and 7 for KAN modified.

While this is not necessarily indicative of a training collapse, especially since the KAN modified class has a more severe bottleneck even though it performed better than the KAN class, it is interesting to note that only the two worst performing classes exhibit this, with the KAN mini parameter size remaining constant, and the MLP gradually varying across layers. It is unclear what could be causing this parameter size collapse. Once possible cause is the Jacobian, which the fourier modules, contains terms of the form:

$$\frac{\partial x_j^{(l+1)}}{\partial x_i^{(l)}} = \sum_{k=1}^{K} \left[A_{ijk}^{(l)} k \cos\left(k x_i^{(l)}\right) - B_{ijk}^{(l)} k \sin\left(k x_i^{(l)}\right) \right] \tag{6.1.1}$$

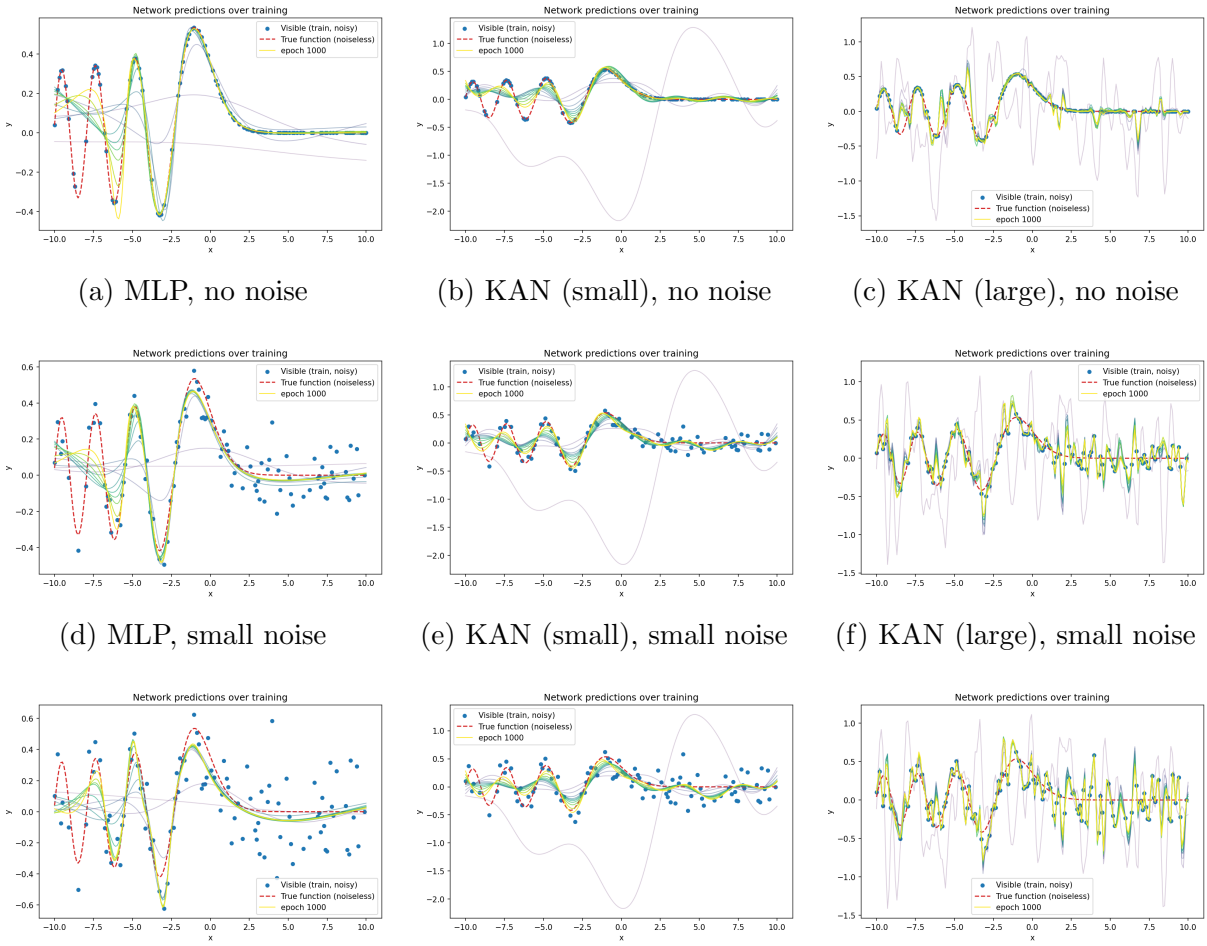
This includes a dependence on the actual layer outputs $x_i^{(l)}$, which, unlike in an MLP, creates gradient dependence on the other parameter values. This more complex loss landscape may be causing the later parameters to face training issues, leading to their reduction in magnitude. This is, however, speculation, and would require further investigation into the behaviour of backpropagation in these deeper networks.

6.2 Overfitting to training data

The nature of the non-linear weight function used in KAN modules intuitively offer increased fitting to complex functions in comparison to MLPs with the same number of parameters. The original KAN paper from Liu et.al. itself notes the ease with which KAN modules can fit closed form expressions, with a two-layer KAN exhibiting similar error to a five-layer MLP containing over 1000 times as many parameters [42]. In the case of CKANGA specifically, the use of a fourier basis, even with a minimal number of harmonics, can take advantage of possible spectral features in the approximated function. This would, anecdotally, further increase the fit of CKANGA to the function being approximated by it. Given the massive size of the original KAN model class, with over two million parameters, there is an inherent risk that the extreme complexity of the network may have overfit the training data. This is a known problem in B-Spline basis KANS [49], which may also carry over to fourier basis KANs. An illustration of the training dynamics on a single variable function for an MLP and two KANs, one smaller and one larger, is seen in figure 6.2.

The KANs appear to have much quicker convergence to their final state after 1000 epochs than the MLP, which takes a few hundred epochs to settle in comparison to the 200 required for the KANs. The larger KAN has significant issues with over fitting compared to both the MLP and the smaller KAN, a feature which appears exacerbated, but not caused by noise in the data. Even with no noise, the larger KAN has issues interpolating data due to overfitting on the available training points, being unable to capture the true values, which are nearly linear interpolations.

While the training datasets are by no means small, they are curated sets of bioactive or drug-like molecules. This means that certain features are naturally more present than others. Lipinski's rule of five, for example, is a qualitative criteria with over thirty five thousand citations that is generally able to differentiate molecules as bioactive, or in other words, their 'drug-likeness'[41]. This criteria imposes limits on certain molecular features, such as length, mass and reactivity. Most chemicals in the ChEMBL small molecules database would be expected to follow these principles, so the training data may be causing the autoencoder to isolate these specific features extremely well. However, on top of the autoencoder never seeing them during training, the domain of each of the test datasets is hyperspecific in comparison in terms of relevant molecular features. While rules such as Lipinski's are suitable for general bioactivity, meaning a molecule that passes the rules will have some effect on the body, the test datasets are for specific biological processes that may not adhere to these more general limitations on molecules. The BACE dataset, for example, is a binary classifier for a molecule interacting with the BACE enzyme, which is a specific target for research into Alzheimer's disease [65]. Through training, the autoencoder may have overfit the contextual bias in the ChEMBL data at higher network



(a) MLP, no noise (b) KAN (small), no noise (c) KAN (large), no noise

(d) MLP, small noise (e) KAN (small), small noise (f) KAN (large), small noise

(g) MLP, large noise (h) KAN (small), large noise (i) KAN (large), large noise

Figure 6.2: Impact of noise level on fitting quality across model architectures. Rows show increasing noise (none, small, large); columns show MLP, small KAN, and large KAN respectively.The MLP consists of a 4 layers network with a hidden width of 64 and tanh activation functions. Small KAN consists of a 2 harmonic, 2 layer network with a hidden width of 16. Large KAN consists of a 3 harmonic, 4 layer network with a hidden width of 32.

complexities. In turn this may have lead to poor encoding of the more domain-specific attributes relevant to the individual test datasets, cascading into poor model performance on the datasets.

6.3 Autoencoder reconstruction loss

The reconstruction loss used in the training of the autoencoder is relatively crude in it's construction. All of the three terms used in the total loss have shortcoming that may have carried errors downstream, particularly in the larger models. Due to the highly variable input size of molecules in the data and in practice, the reconstruction loss was constructed with both input dimension and permutation invariance, hence why a direct reconstruction of the input graphs was not deemed possible without crude padding.

6.3.1 Graph-level term

The graph-level feature term itself is extremely simple, being the sum of all the embedded node vectors, which were the node vectors concatenated with their average feature vectors. This process is extremely reductive in the importance associated with each feature, providing no weighting to more chemically important features. For example, consider this term for the L-DOPA and D-DOPA molecules discussed in the Section 2.3. The signal of the critical stereochemistry difference becomes hidden in a few dimensions of the sum of the node embeddings, meaning that reconstruction is likely to not sufficiently penalise the ignorance of the feature. This in turn leads to a reduced ability to effectively encode minor values changes that may have major chemical importance. This would, in theory, affect all models, and would not be the likely cause for the larger model classes. In fact, the possible overfitting of these models mentioned earlier would counter this effect, as the larger models would intuitively be able to capture these minor differences more effectively.

A potential improvement to this reconstruction term would be to assign individual weightings to these reconstruction features either through expert domain knowledge, or through machine learning methods that could be tuned for autoencoder use in less general molecular embedding applications, such as for specific disease-focused drug screening.

6.3.2 Topological term and co-spectrality

The topological loss term is also a potential source of error in the architecture design. The use of a fixed number of eigenvalues of the laplacian of the molecular graph's adjacency matrix overcomes the issue of variable sized input graphs without the need for padding. While practically effective, there are two primary issues with this approach. Firstly, the requirement for ten non-trivial eigenvalues restricts the input molecules to having at least eleven atoms. While the majority of potential drugs will satisfy this criteria, with the average small molecule drug having around forty atoms [19], there are notable examples of rarer drugs with less than eleven atoms, such as the blood cancer suppressant hydroxyurea. Critically, there are also some which exist within the test datasets that required removal during preprocessing. The nature of this topological term therefore limits the scope of molecules that the can be embedded without padding. While this can be overcome by choosing fewer eigenvalues, such as setting the number of eigenvalues to the size of the smallest molecule of interest, this reinforces the secondary issue of the term in that detailed topological information is naturally lost in what is essentially a dimensional reduction of the graph laplacian. The fixing of the number of eigenvalues used excludes the larger eigenvalues of the graph leading to direct information loss in this regard; however, the eigenvalue decomposition itself is not entirely preserving of the graph topology. The eigenvalues of a given laplacian (the spectrum) are generally non-unique, with graphs sharing the same spectra termed 'co-spectral' [64]. Complete graphs, with edges between all nodes, have unique spectra, yet this not the case in any molecules

aside from the simplest, such as the common structure of oxygen. Trees, graphs with no cycles that characterise many molecules such as fats and common hydrocarbon fuels, are known to asymptotically approach a co-spectral proportion of 100% as the number of nodes increases. This means that large molecules with no ring structures are difficult, if not impossible, to distinguish form other similar molecules topologically based on their laplacian eigenvalues. While ring structures are present in around 95% of drug molecules [5], the use of CKANGA in non-pharmacological contexts that do deal with long-chain molecules is further disparaged by this fact.

This issue of co-spectrality of molecules is documented in literature as 'Isospectral Molecules' [24].

It is not immediately clear how to better encode the topology of the molecular graphs other than maximising the number of eigenvalues based on the data being used. Similar to the suggested improvements for the graph-level loss term, weighting could be placed on each eigenvalue either using prior knowledge or through machine learning techniques. Given an ordered list of a laplacian's eigenvalues from smallest to largest, each eigenvalue has a specific geometric meaning attached to it. For example, the second smallest eigenvalue of the laplacian, including trivial ones, is known as the Fiedler value, and relates to the algebraic connectivity of the graph [15]. Leveraging these properties further in the specific context of molecular embedding may yield additional improvements to the topological aspect of the CKANGA embedding.

6.3.3 Contrastive term and batch-wise similarity

Another potential drawback with the CKANGA architecture is the use of the contrastive learining term. At face value, the contrastive term serves to produce a 'smoother' latent space, embedding graphs with higher similarity metrics closer together in terms of euclidean distance, whilst pushing dissimilar embeddings apart. All graphs, unless of perfect similarity, are also pushed apart slightly up to a given radius in order to prevent dense clustering. The intent of this is to aid in preserving graph similarity into the embedding space, using euclidean distance within this space as a proxy for similarity between two embeddings. The contrastive approach effectively increases the separability of the embedding space, meaning downstream predictions tasks are theoretically better able to discriminate features of the a given embedding.

In the molecular embedding context, the Tanimoto similarity can be calculated on the Morgan fingerprints of two given molecules, as was the approach taken within this thesis. The Morgan fingerprint, itself a chemical embedding, although derived programmatically rather than learnt, encodes the presence of arbitrary substructures of a molecule into binary values stored in vector [51]. While common in machine learning contexts due to their relative ease of computation, Morgan fingerprints are subject to bit-collisions [50] and do

not account for steric differences, which, as discussed in earlier sections, may be critical in predicting molecular interactions. The more pertinent issue with the contrastive methods used in CKANGA is the batchwise application of the similarity metric. Tanimoto similarity is a pairwise scoring, assessing the agreeance of two molecules' Morgan fingerprints. In smaller datasets, this may be fine, as these values can be easily stored in triangular matrices. Loss terms can then compare the given batch similarities against the entire embedded training landscape if all embeddings are stored and updated throughout the training process. This allows a metrics to be compared across the entirety of the training data to ensure the full similarity landscape is being encoded into the latent space. However, this becomes computationally impractical once dataset sizes increase, such as in the case of massive context windows like the ChEMBL datasets. Storing the embeddings and calculating similarity scores at each update would require iterating through over one million values at each individual training step. This would be practically impossible to implement on an effective timescale. Instead, a batch-wise similarity approach was taken. This limits the size of similarity score calculations to the square of the size of the batch, and prevents the need to store embeddings, as they are directly calculated during the forward pass of each batch. While this approach provides a much more practically relevant implementation of contrastive learning, there are additional issues that may arise.

The heterogeneity of the random samples that make up the batches can act severely nullify any effects of the contrastive loss term. Consider, for example, batches that are constituted of two entries, that each belong in two different general clusters or categories. In the batch-wise contrastive term, the two entries in each batch may consistently have low similarity scores. If the embedding space already picks up on key differences between the two entries and separates them in the latent space, then the contrastive term does not act strongly. This is the expected behaviour of the term, but the issue lies in the subsequent inaction within the individual clusters. While the autoencoder may learn to separate out the two general classes, it lacks the contrastive mechanism acting within classes that encodes finer details that may be more critical in downstream prediction than the general category of the entry. A conceptual visualisation of this effect in two-dimensions is given in Figure 6.3.

It is unlikely that this worst case scenario will occur, especially on massive datasets, however, on the balance of probability, it is almost guaranteed to occur in some batches if they are generated through random partitions.

This is a known issue in batch-wise contrastive learning approaches, having a significant impact to model performance [70] [69]. There are several approaches to overcoming this issue, mainly involving the non-random generation of batches [70], or the mixing of batches during training [69]. Both approaches effectively push entries that are more difficult to

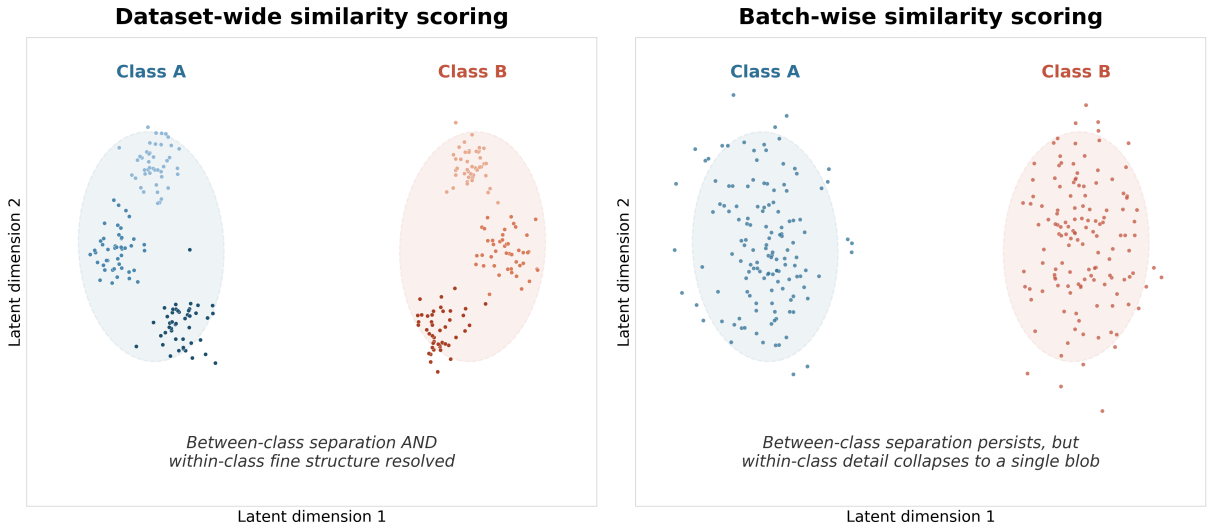


Figure 6.3: A visual explanation of the potential inability for batch wise contrastive learning to capture fine grained detail. Given heterogeneous batches containing entries primarily in different general clusters, the batch-wise approach is unable to apply the contrastive term effectively within these clusters, meaning that finer details that may separate out substructures are not getting learnt by the model

distinguish from one another into the same batch, in turn allowing the contrastive term to better distinguish subtle differences between them during training. The adoption of such techniques would be a welcome addition to improve the CKANGA architecture.

Another issue with the specific approach used is the positive similarity term $T_{i,j} \times d_{i,j}^2$. This term will always act unless the similarity score is strictly zero, giving a form of 'gravity' to every embedded entry that pulls them all together. In itself, this is not a problem. The more compact latent space provides a smoother generalisation to unseen entries, which is critical in molecular embedding tasks such as drug discovery, where the goal is to estimate interactions of novel molecules [26]. However, the effect of this attraction term is not bounded by distance like the $(1 - T_{i,j}) \times (\max(0, m - d_{i,j})^2$ repulsion term. The key variable in these terms is the euclidean distance $d_{i,j}$, which introduces issues with increasing latent space dimensions. In high dimensions, the property of euclidean distance loses a significant amount of meaning, in that the ratio of the largest distance between points and the smallest distance tends to 1 [1]. Not only does this comparative aspect degrade, but the magnitude of distance also increases in higher dimensions. This means that higher dimensional latent spaces will generally have both homogeneous distances between embedded values, and massive attraction terms that likely dominate the loss function.

This effect can actually be seen within the original KAN model class. During hyperparameter tuning, the greatest available latent space size of 512 was preferred, however the stratified tuning approach meant that the contrastive parameters were iterated over last.

The weight assigned to the contrastive term was the lowest value available at 0.1, while the margin term selected was the largest at 4. From a qualitative standpoint, this could be attributed to the high-dimensional latent space causing dominance of the contrastive term, leading the model to perform better with a lower weighting. Moreover, the larger margin term could be seen as the model attempting to maximise the repulsion effect in order to counteract the massive attraction term. The original KAN was the worst performing model class, and while this cannot entirely be attributed to the high dimensional latent space's interaction with the contrastive term, the subsequent 'KAN modified' model class, utilising the same smaller latent space as the MLP model class, demonstrated higher or equal performance across the board. Moreover, with the complete removal of the contrastive term in the 'KAN mini' model class, the model class outperformed all others, although as discussed, this is likely due to other factors such as overfitting or training instability in the larger models. Nonetheless, the current implementation of the contrastive term is likely to impact CKANGA models with high dimensional latent spaces.

Alternative approaches to the contrastive term that could be used to improve CKANGA include the bounding of the attraction term with a tunable margin in a similar way to the repulsion term in order to limit the impact of massive distance values on the overall loss. The use of an alternate distance norm, such as a scaled Manhattan L_1 norm or fractional L_p norms [1], could overcome this dimensionality entirely, and would be highly recommended for any future implementations over the standard euclidean L_2 norm.

6.4 Performance of Small KANs

While there are numerous areas for improvement in the CKANGA architecture as used in this thesis, the most striking result is the performance of the smallest 'KAN mini' model class. At face value, this model class, with a number of parameter on the order of thousands compared to the millions of the full KAN and no contrative element, should have performed significantly worse, as it should have been less 'expressive' with fewer paramenters. However, the opposite result was obtained. This bare-bones version of the CKANGA architecture was the only model to outperform, let alone match, the more conventional CMLPGA, and exhibited very near SOTA performance on the BBBP dataset, especially when considering the simple molecular feature generation used in comparison to richer features used in these SOTA models. This indicates a potential for the 'KAN mini' models to rival the SOTA models in performance when exposed to richer molecular features, however this remained out of scope for this project given computational limitations.

'KAN mini' likely benefited from a variety of changes to the original KAN models. The smaller hidden width, latent space, number of harmonics and number of layers in comparison to the other KANs likely avoided the overfitting problem discussed earlier. On

top of this, the removal of the contrastive term may have been beneficial, however given the reduced embedding space dimensions, it is not clear if this smaller model would be better positioned against the dimensionality issues of the contrastive term. Further experimentation is required in this aspect. The model also had almost certainly more stable training dynamics than it's larger counterparts, as indicated by the consistent parameter size in Figure 6.1 and the overall model performance.

This result demonstrate the potential use case of KANs not as large, massively expressive replacements for deep, traditionally MLP based networks, but as smaller alternatives to large MLPs that can outperform with significantly fewer parameters. Assuming an operation of sine or cosine costs 50 floationg point operations per second (FLOPs), 'KAN mini' would have nearly ten times the FLOPs of the CMLPGA model class, while only having one quarter then number of parameters. There is a clear tradeoff in the use of KANs compared to MLPs, where a KAN, having significantly fewer parameters, may be more suitable to locally deployment situations, whereas the quicker runtime of an MLP could be more suited to cloud based applications.

In the context of molecular embeddings, there are very few situations where a locally deployed network would be required. There is a potential use case in preliminary analytical devices for field-based natural product screening, where a researcher may want to analyse chemical compounds where they are found, however it is unrealistic that such a use is worthy of development given such compounds simply be stored and taken back to labs where larger and faster models could be used. However, KANs may have potential in other fields, where local models are important but runtimes are not, such as in non-real time remote monitoring instruments, or non-emergency medical field clinics. More research is required to bolster the understanding of KANs, but it is clear that use cases are ultimately more niche than was had made out by the seminal paper [42].

Chapter 7

Conclusion

This thesis set out to determine whether Kolmogorov-Arnold Network (KAN) modules could enhance the quality of a Graph Autoencoder's molecular embedding space, addressing a gap in the literature in which KANs had been applied to Graph Convolutional and Graph Attention Networks for molecular property prediction [36], but never to the GAE architecture. To this end, the Contrastive Kolmogorov-Arnold Network Graph Autoencoder (CKANGA) was developed, replacing all MLP modules within a standard GAE with Fourier-basis KAN layers, and trained on subsets of the ChEMBL small molecules database [74] before being benchmarked against an equivalently specified MLP-based autoencoder across the BBBP, HIV, BACE and Tox21 MoleculeNet datasets [67].

The results obtained diverge sharply from conventional expectations of neural network scaling [47]. The original, full-complexity CKANGA architecture, despite possessing over two million parameters, orders of magnitude more than the minimal-complexity variant ultimately setting the bar for performance, consistently underperformed the MLP baseline, in several cases producing embeddings no more useful than random guessing. Systematically reducing the complexity of the architecture, first by matching it's hidden width and latent size to the MLP baseline, and subsequently by removing the contrastive loss term and reducing all remaining structural parameters to their minimum, produced a consistent improvement in performance. The resulting minimal architecture, 'KAN mini', outperformed the MLP baseline across virtually all datasets and pretraining context sizes, and approached reported state-of-the-art performance [36] on the BBBP dataset despite relying on a considerably reduced molecular feature set.

Several candidate explanations for this inverse relationship between complexity and performance were examined. Evidence of parameter magnitude collapse in the deeper layers of the larger KAN-based models pointed to possible training instability [56] [57], plausibly linked to the more complex gradient dependencies introduced by the Fourier KAN modules relative to an MLP. The greater local expressivity of KAN modules [42] also raises the risk of overfitting, both to the specific training points available and to general

drug-likeness characteristics [41] of the ChEMBL pretraining data, at the expense of the more disease-specific features relevant to individual benchmark datasets [65]. Separately, the reconstruction loss terms used to train the autoencoder were shown to be somewhat crude, while the batch-wise formulation of the contrastive loss term was shown to be vulnerable to both unrepresentative batch composition [70, 69] and the degradation of Euclidean distance as a meaningful metric in higher-dimensional latent spaces [1].

Taken together, these findings suggest that the value of KAN-based architectures in molecular embedding, and potentially more broadly, may lie less in their capacity to serve as large, highly expressive replacements for deep MLP-based networks, and more in their ability to match or exceed MLP performance at a small fraction of the parameter count. This is a narrower role than that implied by the broader KAN literature [42], and one possibly better suited to resource constrained deployment contexts than to large-scale, general-purpose molecular embedding pipelines. These conclusions should, however, be tempered by the reduced molecular feature set and the computational constraints placed on the hyperparameter tuning process used throughout this thesis, both of which limit the extent to which the results can be generalised.

Future work motivated by these findings includes the introduction of domain-informed or learned feature weighting into the reconstruction loss terms, improved topological encoding strategies that make better use of the structural meaning attached to individual Laplacian eigenvalues [15], non-random or mixed batch construction strategies [70] [69], alternative distance formulations [1] for the contrastive loss term, and, most notably, an evaluation of the 'KAN mini' architecture using the richer molecular feature sets employed by current state-of-the-art models [36], to determine whether it's promising performance under a reduced feature set can be extended to match or exceed existing benchmarks.

Chapter 8

Appendix

8.1 Molecular feature generation

Table 8.1: Node and edge features used in molecular graph construction. Each feature is listed with it's associated data type and number of dimensions.

Type	**Feature**	**Type**	**# Dim**
Node Features (23)			
Node	Atomic number	Integer	1
Node	Number of bonded neighbours (degree)	Integer	1
Node	Hybridisation (SP, SP2, SP3, SP3D, SP3D2, Other)	One-hot binary	6
Node	Formal charge	Integer	1
Node	Implicit valence	Integer	1
Node	Number of implicit hydrogens	Integer	1
Node	Ring membership	Binary	1
Node	Aromaticity	Binary	1
Node	Chirality (CW, CCW, Other)	One-hot binary	3
Node	Atomic mass	Continuous	1
Node	Number of radical electrons	Integer	1
Node	Explicit valence	Integer	1
Node	van der Waals radius	Continuous	1
Node	Covalent radius	Continuous	1
Node	Hydrogen bond acceptor	Binary	1
Node	Hydrogen bond donor	Binary	1
Edge Features (10)			
Edge	Bond order	Integer	1
Edge	Conjugated bond	Binary	1
Edge	Ring membership	Binary	1
Edge	Bond stereochemistry (E, Z, Other)	One-hot binary	3
Edge	Bond length	Continuous	1
Edge	Inverse square of bond length ($1/d^2$)	Continuous	1
Edge	Strained bond/Small-ring indicator (3-4 members)	Binary	1
Edge	Linear bond indicator (SP hybridised)	Binary	1

8.2 Dataset summary

Table 8.2: Summary of the MoleculeNet datasets used for model evaluation. The recommended split is the MoleculeNet benchmark's choice of split for the given dataset. For Tox21, although a Random split is recommended, a Scaffold Split was conducted for this thesis

Dataset	**Application**	**Size**	**Prediction Task**	**Recommended Split**
BACE	Prediction of inhibitor activity against human β-secretase 1 (BACE-1), an Alzheimer's disease drug target.	1,522	Binary	Scaffold
BBBP	Prediction of blood–brain barrier permeability for central nervous system drug candidates.	2,053	Binary	Scaffold
HIV	Prediction of a compound's ability to inhibit HIV replication.	41,913	Binary	Scaffold
Tox21	Prediction of toxicity across 12 biological assays involving nuclear receptors and stress response pathways.	8,014	12-Task Binary	Random (Scaffold Used)

8.3 Results tables

Table 8.3: KAN encoder results across MoleculeNet datasets (AUC-ROC over $n = 100$ runs).

Dataset	**Mean**	**Std**	**Min**	**Max**	**<0.5**	n
200K						
BACE	0.5564	0.0209	0.5159	0.6667	0	100
BBBP	0.5242	0.0324	0.4328	0.5999	20	100
HIV	0.5476	0.0218	0.4953	0.5909	2	100
TOX21	0.5884	0.0069	0.5680	0.6060	0	100
500K						
BACE	0.4876	0.0403	0.4097	0.6056	68	100
BBBP	0.5160	0.0391	0.4468	0.6062	43	100
HIV	0.5241	0.0191	0.4891	0.5763	9	100
TOX21	0.5926	0.0080	0.5763	0.6122	0	100
1M						
BACE	0.5401	0.0494	0.4374	0.6533	23	100
BBBP	0.5269	0.0380	0.4453	0.6463	28	100
HIV	0.5316	0.0192	0.4973	0.5855	1	100
TOX21	0.5945	0.0086	0.5736	0.6116	0	100

Table 8.4: KAN modified encoder results across MoleculeNet datasets (AUC-ROC over $n = 100$ runs).

Dataset	**Mean**	**Std**	**Min**	**Max**	**<0.5**	n
200K						
BACE	0.6418	0.0249	0.5789	0.6963	0	100
BBBP	0.5949	0.0350	0.4837	0.7205	1	100
HIV	0.6812	0.0150	0.6535	0.7210	0	100
TOX21	0.5954	0.0228	0.5368	0.6470	0	100
500K						
BACE	0.6707	0.0521	0.2992	0.7087	2	100
BBBP	0.5897	0.0111	0.5787	0.6694	0	100
HIV	0.6213	0.0040	0.6165	0.6363	0	100
TOX21	0.5506	0.0279	0.4977	0.6180	1	100
1M						
BACE	0.6799	0.0071	0.6684	0.7067	0	100
BBBP	0.5999	0.0053	0.5886	0.6151	0	100
HIV	0.6509	0.0018	0.6491	0.6636	0	100
TOX21	0.5357	0.0230	0.4977	0.6028	1	100

Table 8.5: KAN mini encoder results across MoleculeNet datasets (AUC-ROC over $n = 100$ runs).

Dataset	**Mean**	**Std**	**Min**	**Max**	**<0.5**	n
200K						
BACE	0.7624	0.0125	0.7376	0.7969	0	100
BBBP	**0.7594**	0.0099	0.7305	0.7910	0	100
HIV	**0.7314**	0.0128	0.7097	0.7776	0	100
TOX21	**0.7481**	0.0036	0.7402	0.7591	0	100
500K						
BACE	0.7917	0.0111	0.7650	0.8209	0	100
BBBP	0.7301	0.0113	0.7060	0.7713	0	100
HIV	0.7189	0.0087	0.6990	0.7395	0	100
TOX21	0.7450	0.0040	0.7342	0.7533	0	100
1M						
BACE	**0.8114**	0.0105	0.7848	0.8349	0	100
BBBP	0.7383	0.0114	0.7084	0.7772	0	100
HIV	0.7300	0.0087	0.7083	0.7631	0	100
TOX21	0.7310	0.0058	0.7140	0.7457	0	100

Table 8.6: MLP encoder results across MoleculeNet datasets (AUC-ROC over $n = 100$ runs).

Dataset	**Mean**	**Std**	**Min**	**Max**	**<0.5**	n
200K						
BACE	0.7281	0.0067	0.7171	0.7509	0	100
BBBP	0.6328	0.0043	0.6264	0.6485	0	100
HIV	0.6748	0.0114	0.6521	0.7042	0	100
TOX21	0.7221	0.0050	0.7092	0.7344	0	100
500K						
BACE	0.7222	0.0134	0.6904	0.7554	0	100
BBBP	0.7399	0.0097	0.7170	0.7674	0	100
HIV	0.6450	0.0125	0.6061	0.6690	0	100
TOX21	0.7200	0.0053	0.7089	0.7323	0	100
1M						
BACE	0.7151	0.0090	0.6936	0.7359	0	100
BBBP	0.7304	0.0136	0.6976	0.7583	0	100
HIV	0.7091	0.0124	0.6762	0.7418	0	100
TOX21	0.7173	0.0042	0.7093	0.7289	0	100

8.4 Hyperparameter Tuning

The following tables outline the results and grids of the stratified tuning process for botht the KAN and MLP models. Tables are given in tuning order, that is, left to right, then top to bottom. Table numericla order also corresponds to tuning order.

8.4.1 KAN Model

Table 8.7: KAN Hyperparameter Tuning: Hidden Width

Hidden Width	**Mean AUC**
32	0.6275
64	0.6240
128	0.6287
256	**0.6324**

Table 8.8: KAN Hyperparameter Tuning: Message Layers

Message Layers	**Mean AUC**
1	0.6324
2	0.6389
3	0.6587
4	**0.6657**

Table 8.9: KAN Hyperparameter Tuning: Harmonics

Harmonics	**Mean AUC**
1	0.6657
2	**0.7320**
3	0.7235
5	0.7199

Table 8.10: KAN Hyperparameter Tuning: Readout Layers

Readout Layers	**Mean AUC**
1	0.7265
2	0.7235
3	**0.7338**

Table 8.11: KAN Hyperparameter Tuning: Latent Size

Latent Size	**Mean AUC**
32	0.6663
64	0.6988
128	0.7338
256	0.7369
512	**0.7588**

Table 8.12: KAN Hyperparameter Tuning: Decoder Layers

Decoder Layers	**Mean AUC**
1	0.7607
2	0.7579
3	**0.7819**
4	0.7626

Table 8.13: KAN Hyperparameter Tuning: Topology Ratio

Topology Ratio	Mean AUC
0.1	0.7915
0.3	**0.7956**
0.5	0.7819
0.7	0.7660
0.9	0.7542

Table 8.14: KAN Hyperparameter Tuning: Learning Rate

Learning Rate	Mean AUC
1e-05	0.6991
0.0001	**0.7956**
0.001	0.6462

Table 8.15: KAN Hyperparameter Tuning: Contrastive Weight

Contrastive Weight	Mean AUC
0.1	**0.7958**
0.2	0.7763
0.3	0.7676
0.4	0.7586
0.5	0.7540

Table 8.16: KAN Hyperparameter Tuning: Margin

Margin	Mean AUC
0.1	0.7895
0.5	0.7839
1.0	0.7956
2.0	0.7947
4.0	**0.8003**

8.4.2 MLP Model

Table 8.17: MLP Hyperparameter Tuning: Hidden Width

Hidden Width	Mean AUC
32	0.6533
64	**0.6573**
128	0.6554
256	0.6564

Table 8.18: MLP Hyperparameter Tuning: Message Layers

Message Layers	Mean AUC
1	0.6552
2	0.6573
3	**0.6600**
4	0.6577

Table 8.19: MLP Hyperparameter Tuning: Readout Layers

Readout Layers	Mean AUC
2	0.6600
3	**0.6602**
4	0.6590

Table 8.20: MLP Hyperparameter Tuning: Latent Size

Latent Size	Mean AUC
32	0.6505
64	0.6594
128	**0.6602**
256	0.6587
512	0.6572

Table 8.21: MLP Hyperparameter Tuning: Decoder Layers

Decoder Layers	Mean AUC
2	**0.6601**
3	0.6563
4	0.6372

Table 8.22: MLP Hyperparameter Tuning: Topology Ratio

Topology Ratio	Mean AUC
0.1	0.6547
0.3	0.6584
0.5	**0.6602**
0.7	0.6513
0.9	0.6120

Table 8.23: MLP Hyperparameter Tuning: Learning Rate

Learning Rate	Mean AUC
1e-05	0.6354
0.0001	0.6602
0.001	**0.6637**

Table 8.24: MLP Hyperparameter Tuning: Contrastive Weight

Contrastive Weight	Mean AUC
0.1	0.6649
0.2	0.6576
0.3	0.6604
0.4	0.6613
0.5	**0.6683**

Table 8.25: MLP Hyperparameter Tuning: Margin

Margin	**Mean AUC**
0.1	0.6137
0.5	0.6412
1.0	**0.6683**
2.0	0.6679
4.0	0.6674

References

[1] Aggarwal, C. C., Hinneburg, A., Keim, D. A., *On the Surprising Behavior of Distance Metrics in High Dimensional Space*, Proceedings of the 8th International Conference on Database Theory (ICDT), Lecture Notes in Computer Science, **1973**, 420–434, 2001.

[2] Aghaei, A. A., *fKAN: Fractional Kolmogorov-Arnold Networks with Trainable Jacobi Basis Functions*, arXiv preprint arXiv:2406.07456, 2024.

[3] Aghaei, A. A., *rKAN: Rational Kolmogorov-Arnold Networks*, arXiv preprint arXiv:2406.14495, 2024.

[4] Alberts, B., Johnson, A., Lewis, J., Raff, M., Roberts, K., Walter, P., *The Shape and Structure of Proteins*, in: *Molecular Biology of the Cell*, 4th edition, Garland Science, New York, 2002.

[5] Aldeghi, M., Malhotra, S., Selwood, D. L., Chan, A. W. E., *Two- and Three-Dimensional Rings in Drugs*, Chemical Biology & Drug Design, **83**(4), 450–461, 2014.

[6] Arnold, V. I., *On functions of three variables*, Doklady Akademii Nauk SSSR, **114**(4), 679–681, 1957.

[7] Arnold, V. I., *On the representation of continuous functions of several variables as superpositions of continuous functions of one variable and addition*, Doklady Akademii Nauk SSSR, **120**(1), 9–12, 1958.

[8] Avendaño, C., Menéndez, J. C., *Other Nonbiological Approaches to Targeted Cancer Chemotherapy*, in: *Medicinal Chemistry of Anticancer Drugs*, 2nd edition, Elsevier, Amsterdam, pp. 493–560, 2015.

[9] Besharatifard, M., Vafaee, F., *A Review on Graph Neural Networks for Predicting Synergistic Drug Combinations*, Artificial Intelligence Review, **57**, 49, 2024.

[10] Botting, J., *The History of Thalidomide*, Drug News & Perspectives, **15**(9), 604–611, 2002.

[11] Bozorgasl, Z., Chen, H., *Wav-KAN: Wavelet Kolmogorov-Arnold Networks*, arXiv preprint arXiv:2405.12832, 2024.

[12] Braun, J., Griebel, M., *On a Constructive Proof of Kolmogorov's Superposition Theorem*, Constructive Approximation, **30**(3), 653–67

[13] Cools, R., *Dopaminergic Modulation of Cognitive Function – Implications for L-DOPA Treatment in Parkinson's Disease*, Neuroscience & Biobehavioral Reviews, **30**(1), 1–23, 2006.

[14] Fakhoury, D., Fakhoury, E., Speleers, H., *ExSpliNet: An Interpretable and Expressive Spline-Based Neural Network*, Neural Networks, **152**, 332–346, 2022.

[15] Fiedler, M., *Algebraic Connectivity of Graphs*, Czechoslovak Mathematical Journal, **23**(2), 298–305, 1973.

[16] Fooladi, H.; Vu, T. N. L.; Mathea, M.; Kirchmair, J. Evaluating Machine Learning Models for Molecular Property Prediction: Performance and Robustness on Out-of-Distribution Data. *J. Chem. Inf. Model.* **2025**, *65*, 9871–9891.

[17] Fridman, B. L., *Improvement in the smoothness of functions in the Kolmogorov theorem on superpositions*, Doklady Akademii Nauk SSSR, **177**(5), 1019–1022, 1967.

[18] Funahashi, K., *On the Approximate Realization of Continuous Mappings by Neural Networks*, Neural Networks, **2**(3), 183–192, 1989.

[19] Ghose, A. K., Viswanadhan, V. N., Wendoloski, J. J., *A Knowledge-Based Approach in Designing Combinatorial or Medicinal Chemistry Libraries for Drug Discovery. 1. A Qualitative and Quantitative Characterization of Known Drug Databases*, Journal of Combinatorial Chemistry, **1**(1), 55–68, 1999.

[20] GistNoesis, *FourierKAN*, GitHub repository, 2024. Available: https://github.com/GistNoesis/FourierKAN

[21] Girosi, F. and Poggio, T., *Representation Properties of Networks: Kolmogorov's Theorem Is Irrelevant*, Neural Computation, vol. 1, no. 4, pp. 465-469, 1989.

[22] Guo, Q.; Hernandez-Hernandez, S.; Ballester, P. J. Scaffold Splits Overestimate Virtual Screening Performance. In *Artificial Neural Networks and Machine Learning – ICANN 2024*; Wand, M., Malinovská, K., Schmidhuber, J., Tetko, I. V., Eds.; Lecture Notes in Computer Science, vol. 15025; Springer: Cham, 2024; pp. 58–72.

[23] Hecht-Nielsen, R., *Kolmogorov's Mapping Neural Network Existence Theorem*, Proceedings of the IEEE International Conference on Neural Networks, **III**, 11–13, 1987.

[24] Herndon, W. C., Ellzey, M. L., *Isospectral Graphs and Molecules*, Tetrahedron, **31**(2), 99–107, 1975.

[25] Hou, Y., Ji, T., Zhang, D., Stefanidis, A., *Kolmogorov-Arnold Networks: A Critical Assessment of Claims, Performance, and Practical Viability*, arXiv preprint arXiv:2407.11075, 2024.

[26] Hughes, J. P., Rees, S., Kalindjian, S. B., Philpott, K. L., *Principles of Early Drug Discovery*, British Journal of Pharmacology, **162**(6), 1239–1249, 2011.

[27] Igelnik, B., Parikh, N., *Kolmogorov's Spline Network*, IEEE Transactions on Neural Networks, **14**(4), 725–733, 2003.

[28] Jia, Z.-C., Yang, X., Wu, Y.-K., Li, M., Das, D., Chen, M.-X., Wu, J., *The Art of Finding the Right Drug Target: Emerging Methods and Strategies*, Pharmacological Reviews, **76**(5), 896–914, 2024.

[29] Kaufman, R., *Linear superpositions of smooth functions*, Proc. Amer. Math. Soc., **46**(3), 360–362, 1974.

[30] Kehe, J., Kulesa, A., Ortiz, A., Blainey, P. C., *Massively Parallel Screening of Synthetic Microbial Communities*, Proceedings of the National Academy of Sciences, **116**(26), 12804–12809, 2019.

[31] Kim, S., Lu, P. Y., Mukherjee, S., Gilbert, M., Jing, L., Čeperić, V., Soljačić, M., *Integration of Neural Network-Based Symbolic Regression in Deep Learning for Scientific Discovery*, IEEE Transactions on Neural Networks and Learning Systems, **32**(9), 4166–4177, 2021.

[32] Kolmogorov, A. N., *On the representation of continuous functions of several variables by superposition of continuous functions of one variable and addition*, Doklady Akademii Nauk SSSR, **114**(5), 953–956, 1957.

[33] Kůrková, V., *Kolmogorov's Theorem Is Relevant*, Neural Computation, **3**(4), 617–622, 1991.

[34] Kůrková, V., *Kolmogorov's Theorem and Multilayer Neural Networks*, Neural Networks, **5**(4), 501–506, 1991.

[35] Lai, M.-J., Shen, Z., *The Kolmogorov Superposition Theorem Can Break the Curse of Dimensionality When Approximating High Dimensional Functions*, arXiv preprint arXiv:2112.09963, 2021.

[36] Li, L., Zhang, Y., Wang, G., Xia, K., *Kolmogorov–Arnold Graph Neural Networks for Molecular Property Prediction*, Nature Machine Intelligence, **7**, 1346–1354, 2025.

[37] Li, Z., *Kolmogorov-Arnold Networks Are Radial Basis Function Networks*, arXiv preprint arXiv:2405.06721, 2024.

[38] Limban, C., Nuţă, D. C., Chiriţă, C., Negreş, S., Arsene, A. L., Goumenou, M., Karakitsios, S. P., Tsatsakis, A. M., Sarigiannis, D. A., *The Use of Structural Alerts to Avoid the Toxicity of Pharmaceuticals*, Toxicology Reports, **5**, 943–953, 2018.

[39] Lin, H. W., Tegmark, M., Rolnick, D., *Why Does Deep and Cheap Learning Work So Well?*, Journal of Statistical Physics, **168**, 1223–1247, 2017.

[40] Lip, S., Padmanabhan, S., *Introduction to Precision Medicine*, Medicine, **53**(7), 476–482, 2025.

[41] Lipinski, C. A., Lombardo, F., Dominy, B. W., Feeney, P. J., *Experimental and Computational Approaches to Estimate Solubility and Permeability in Drug Discovery and Development Settings*, Advanced Drug Delivery Reviews, **46**(1–3), 3–26, 2001.

[42] Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljačić, M., Hou, T. Y., Tegmark, M., *KAN: Kolmogorov-Arnold Networks*, arXiv preprint arXiv:2404.19756, 2024.

[43] Long, L., *KA-GNN: Kolmogorov-Arnold Graph Neural Networks*, GitHub repository, 2024. Available: https://github.com/LongLee220/KA-GNN

[44] Lorentz, G. G., *Metric Entropy, Widths, and Superposition of Functions*, The American Mathematical Monthly, vol. 69, no. 6, pp. 469-485, 1962.

[45] Michaud, E. J., Liu, Z., Tegmark, M., *Precision Machine Learning*, Entropy, **25**(1), 175, 2023.

[46] Mswahili, M. E., Jeong, Y.-S., *Transformer-Based Models for Chemical SMILES Representation: A Comprehensive Literature Review*, Heliyon, **10**(20), e39038, 2024.

[47] Nakkiran, P., Kaplun, G., Bansal, Y., Yang, T., Barak, B., Sutskever, I., *Deep Double Descent: Where Bigger Models and More Data Hurt*, International Conference on Learning Representations (ICLR), 2020.

[48] Riederer, P., Strobel, S., Nagatsu, T., Watanabe, H., Chen, X., Löschmann, P.-A., Sian-Hulsmann, J., Jost, W. H., Müller, T., Dijkstra, J. M., Monoranu, C.-M., *Levodopa Treatment: Impacts and Mechanisms Throughout Parkinson's Disease Progression*, Journal of Neural Transmission, **132**(6), 743–779, 2025.

[49] Rigas, S., Verma, D., Alexandridis, G., Wang, Y., *Initialization Schemes for Kolmogorov-Arnold Networks: An Empirical Study*, International Conference on Learning Representations (ICLR), 2026.

[50] Riniker, S., Landrum, G. A., *Similarity Maps – A Visualization Strategy for Molecular Fingerprints and Machine-Learning Methods*, Journal of Cheminformatics, **5**(1), 43, 2013.

[51] Rogers, D., Hahn, M., *Extended-Connectivity Fingerprints*, Journal of Chemical Information and Modeling, **50**(5), 742–754, 2010.

[52] Roque, L., Soares, C., Cerqueira, V., Torgo, L., *Cherry-Picking in Time Series Forecasting: How to Select Datasets to Make Your Model Shine*, Proceedings of the 39th AAAI Conference on Artificial Intelligence, 2025.

[53] Schmidt-Hieber, J., *The Kolmogorov–Arnold Representation Theorem Revisited*, Neural Networks, **137**, 119–126, 2021.

[54] Shen, Z., Yang, H., Zhang, S., *Neural Network Approximation: Three Hidden Layers Are Enough*, Neural Networks, **141**, 160–173, 2020.

[55] Sidarth, SS., Keerthana, A. R., Gokul, R., Anas, K. P., *Chebyshev Polynomial-Based Kolmogorov-Arnold Networks: An Efficient Architecture for Nonlinear Function Approximation*, arXiv preprint arXiv:2405.07200, 2024.

[56] Sohail, S., *On Training of Kolmogorov-Arnold Networks*, arXiv preprint arXiv:2411.05296, 2024.

[57] Spotorno, E. N., Leal Filho, J., Fröhlich, A. A., *Empirical Stability Analysis of Kolmogorov-Arnold Networks in Hard-Constrained Recurrent Physics-Informed Discovery*, arXiv preprint arXiv:2602.09988, 2026.

[58] Sprecher, D. A., *On the Structure of Continuous Functions of Several Variables*, Transactions of the American Mathematical Society, vol. 115, pp. 340–355, 1965.

[59] Ta, H.-T., *BSRBF-KAN: A Combination of B-Splines and Radial Basis Functions in Kolmogorov-Arnold Networks*, arXiv preprint arXiv:2406.11173, 2024.

[60] Tabana, Y., Babu, D., Fahlman, R., Siraki, A. G., Barakat, K., *Target Identification of Small Molecules: An Overview of the Current Applications in Drug Discovery*, BMC Biotechnology, **23**, 44, 2023.

[61] Thalidomide Trust, *About Thalidomide*, Thalidomide Trust, Available: https://thalidomidetrust.org/about-us/about-thalidomide/, Accessed: 1 July 2026.

[62] Tunyasuvunakool, K., Adler, J., Wu, Z., Green, T., Zielinski, M., Žídek, A., Bridgland, A., Cowie, A., Meyer, C., Laydon, A., Velankar, S., Kleywegt, G. J., Bateman, A., Evans, R., Pritzel, A., Figurnov, M., Ronneberger, O., Bates, R., Kohl, S. A. A., Potapenko, A., Ballard, A. J., Romera-Paredes, B., Nikolov, S., Jain, R., Clancy,

E., Reiman, D., Petersen, S., Senior, A. W., Kavukcuoglu, K., Birney, E., Kohli, P., Jumper, J., Hassabis, D., *Highly Accurate Protein Structure Prediction for the Human Proteome*, Nature, **596**(7873), 590–596, 2021.

[63] University of New South Wales, *Katana*, Research Data Australia, DOI: 10.26190/669X-A286, Available: https://researchdata.edu.au/katana/1733007, Accessed: 2026.

[64] van Dam, E. R., Haemers, W. H., *Which Graphs Are Determined by Their Spectrum?*, Linear Algebra and its Applications, **373**, 241–272, 2003.

[65] Vassar, R., Kuhn, P.-H., Haass, C., Kennedy, M. E., Rajendran, L., Wong, P. C., Lichtenthaler, S. F., *Function, Therapeutic Potential and Cell Biology of BACE Proteases: Current Status and Future Prospects*, Journal of Neurochemistry, **130**(1), 4–28, 2014.

[66] Vitushkin, A. G. and Henkin, G. M., *Linear superpositions of functions*, Russian Math. Surveys, **22**(1), 77–126, 1967.

[67] Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leswing, K., Pande, V., *MoleculeNet: A Benchmark for Molecular Machine Learning*, arXiv preprint arXiv:1703.00564, 2017.

[68] Xu, J., Chen, Z., Li, J., Yang, S., Wang, W., Hu, X., Ngai, E. C. H., *FourierKAN-GCF: Fourier Kolmogorov-Arnold Network – An Effective and Efficient Feature Transformation for Graph Collaborative Filtering*, arXiv preprint arXiv:2406.01034, 2024.

[69] Yan, X., Zheng, R., Wu, F., Li, M., *CLAIRE: Contrastive Learning-Based Batch Correction Framework for Better Balance Between Batch Mixing and Preservation of Cellular Heterogeneity*, Bioinformatics, **39**(3), btad099, 2023.

[70] Yang, Z., Huang, T., Ding, M., Dong, Y., Ying, R., Cen, Y., Geng, Y., Tang, J., *BatchSampler: Sampling Mini-Batches for Contrastive Learning in Vision, Language, and Graphs*, Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2023.

[71] Yarotsky, D., *Elementary Superexpressive Activations*, Proceedings of the 38th International Conference on Machine Learning, PMLR, **139**, 11932–11940, 2021.

[72] Yu, R., Yu, W., Wang, X., *KAN or MLP: A Fairer Comparison*, arXiv preprint arXiv:2407.16674, 2024.

[73] Yu, T., Qiu, J., Yang, J., Oseledets, I., *Sinc Kolmogorov-Arnold Network and Its Applications on Physics-Informed Neural Networks*, arXiv preprint arXiv:2410.04096, 2024.

[74] Zdrazil, B., Felix, E., Hunter, F., Manners, E. J., Blackshaw, J., Corbett, S., de Veij, M., Ioannidis, H., Mendez Lopez, D., Mosquera, J. F., *The ChEMBL Database in 2023: A Drug Discovery Platform Spanning Multiple Bioactivity Data Types and Time Periods*, Nucleic Acids Research, **52**(D1), D1180–D1192, 2024.

[75] Zhang, J. et al., *Kolmogorov-Arnold Fourier Networks*, arXiv preprint arXiv:2502.06018, 2025.

[76] Zheng, L. N., Zhang, W. E., Yue, L., Xu, M., Maennel, O., Chen, W., *Free-Knots Kolmogorov-Arnold Network: On the Analysis of Spline Knots and Advancing Stability*, arXiv preprint arXiv:2501.09283, 2025.

[77] Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M., *Graph Neural Networks: A Review of Methods and Applications*, AI Open, **1**, 57–81, 2020.

The following AI tools were used in at least one aspect of this thesis:

- Anthropic. 2026, Claude Sonnet 5 [Large Language Model].

  Retrieved from https://claude.ai/

- OpenAI. 2026, ChatGPT [Large Language Model].

  Retrieved from https://chatgpt.com/

The use of these AI tools within this thesis are limited to:

- Broad initial literature reviews
- Debugging of programming scripts through provision of error statements to models
- Identification of efficiency improvements in programming scripts through descriptions of code workflows to models.
- The generation of code scaffolds for Figures 5.1, 5.2, 5.3, 5.4, 6.2 and 6.3 through direct prompts for specific figure types in the code environment of choice.
- The generation of latex scaffolds for formatting of all tables shown within the thesis through the provision of results data to the models
- The generation of synthetic toy data for Figure 6.3

The above use cases were checked for possible errors, inaccuracies, and bias. Changes were made where necessary to ensure the submitted work is my own, within the guidelines of permitted AI use.

Generative AI assistance was not used in any aspect of the design or writing of this thesis, nor in any aspect the creation of any of the figures not listed above. All conclusions, stylistic voice, speculations, and arguments are my own.